Comparative Evaluation of Constrained and Unconstrained Deep Reinforcement Learning for Safe Robotic Grasping on a UR5 Manipulator in Simulation

Md. Abdur Rabbi

Roll No.: 2031023

Date of Defense: 08 August, 2026

Department of Mechatronics Engineering

Khulna University of Engineering & Technology

Khulna-9203, Bangladesh

CHAPTER 1

INTRODUCTION

A manipulator is, in its simplest description, a chain of rigid links connected by powered joints, ending in a tool that does the actual work of picking something up, placing it, or holding it against a process. For most of industrial robotics' history this chain worked alone, fenced off from people, repeating a single memorised path thousands of times a shift. The collaborative arm changes that arrangement: it is built to share a workspace with a person, which means every motion it makes must be treated as something that could come near a hand, not just near a workpiece. This thesis is built around one specific arm of that kind, and the platform is not incidental to the argument. Everything the simulation trains against, from the joint speeds it is allowed to command to the distances it is allowed to close, is set to match what this particular machine can actually do.

Figure 1.1. A UR5e collaborative arm sharing a workspace with a person. Photograph reproduced from Xia et al. [1], Fig. 1, under that paper's CC BY-NC-ND 4.0 licence. Placeholder pending an original photograph of the platform used in this project.

The platform is a Universal Robots UR5e, a six-joint (6-DOF) collaborative arm [2]. It carries a payload of 5 kg to a reach of 850 mm, repeats a taught position to within 0.03 mm, and weighs 20.6 kg unmounted. Every one of its six joints is rated to the same maximum speed, 180 deg/s, which is exactly π rad/s. The simulated arm in this thesis is capped at 3.14 rad/s for the same reason a real one is: past that speed the motion stops being something a person nearby could react to in time. The provenance of that figure matters: 3.14 rad/s

is the datasheet limit itself, carried into the simulator so that a policy trained there is never asked to move faster than the physical machine could follow it. The UR5e also ships with seventeen configurable safety functions and is certified to the relevant collaborative-robot safety standards, ISO 13849-1 (Category 3, Performance Level d) and ISO 10218-1 [2]. None of those seventeen functions is exercised by the simulation. What this thesis borrows from the platform is its physical envelope, its speed and reach limits, not its built-in safety controller, and Chapter 3 is explicit about which of the two is under test.

Machines with this specification already do real work outside the laboratory. UR-class arms in the UR5e's size class are common on production floors for exactly the tasks a 5 kg payload and an 850 mm reach suit: pick-and-place, machine tending, small-parts assembly and packaging, and inspection stations where a part has to be picked up and held in front of a camera. The same collaborative certification that lets one work next to a person on a line is also why it turns up so often in research: it is affordable enough for a university laboratory, safety-rated enough to run near people without a fenced cell, and common enough on real production floors that a result obtained on one arm should say something useful about the rest of that population. Two of the papers already cited in this chapter used a UR5-class arm for exactly that reason, one for grasping [3] and one for collision avoidance around a moving person [1]. The platform used here is drawn from that same working population.

Two further reasons fixed the choice. The preceding project in this laboratory used a different, custom five-degree-of-freedom arm for classical visual servoing [4]; this thesis moves to a UR-class platform because the safety question it asks, how a learned policy behaves near a joint limit or a kinematic singularity, is best posed on hardware whose limits and safety record are already public and well documented. Isaac Lab, the simulation framework used throughout (Chapter 3), also ships the UR5e as one of its reference manipulators, so building the environment on this arm meant working from a supported example rather than adapting a custom robot from scratch. The joint limits and speed ceiling used throughout this thesis therefore belong to a machine that exists, not one assumed for convenience.

1.1 Background

Industrial manipulators increasingly work in spaces shared with people rather than behind a fence, and the UR5e described above is a working example of that shift. A controller that follows a fixed, hand-derived trajectory can be inspected and bounded before it ever moves: an engineer can trace every configuration it will visit and check each one against a joint limit or a known obstacle. A policy obtained by training a reinforcement learning agent cannot be inspected the same way. Its behaviour is known only through the statistics of the trajectories it produces during and after training, not through a proof that covers every configuration it might reach. Safe operation of a UR-class collaborative arm around people

is already treated as a live industrial requirement, not a hypothetical one [1]. Reinforcement learning is attractive for manipulation precisely because it can learn grasping and placement behaviour directly from experience, without a hand-built controller for every object and pose the arm might encounter. What that learned behaviour does in the configurations it visits rarely, however, is not a detail to be checked after the fact. For a policy sharing a workspace with a person, it is the condition deployment depends on.

The standard formulation of reinforcement learning gives a designer only one channel for stating what the agent should avoid: the reward function. Take the UR5e's own joint-speed ceiling as a concrete case. A designer who wants the trained policy to respect it has, in the standard formulation, exactly one lever: add a penalty term for exceeding it, sized by a weight that trades a unit of excess speed against a unit of task reward. That weight has to be fixed before training starts, before the designer has any real sense of how large a violation the untrained policy will produce or how much task performance the penalty will cost once it starts to bind. Set the weight too small and the constraint is decorative. Set it too large and the agent may refuse to move at all. An alternative, pursued in this thesis, is to keep the requirement outside the reward entirely and state it instead as a constraint with an explicit budget: a quantity that can be set from a measurement of the system, such as the arm's own rated limits, and checked afterwards against what the trained policy actually used [5]. The distinction matters because of what each side has to answer for. A reward weight is a guess, tuned by trial and error against whatever the agent happens to do. A constraint budget is a number the designer can defend before training ever starts, because it comes from a measurement rather than a hunch. Chapter 2 reviews this distinction, and the literature built on either side of it, in full.

1.2 Problem Description

This thesis puts that alternative to a direct test. It trains a UR5e in simulation to pick up and lift a cube, comparing a standard proximal policy optimisation (PPO) agent against a constrained variant of the same algorithm (PPO-Lagrangian) under a shared floor on manipulability, a measure of how far a configuration sits from a kinematic singularity. Such a comparison, as usually run in the literature, has two problems.

- **The cost-critic confound.** Adding a constraint to an RL agent does not add only a constraint. A Lagrangian method also adds a cost critic, a second value network trained alongside the policy to estimate the constraint violation the current policy would incur. That network is present whether or not the constraint is actively binding, and its presence can influence training through machinery the two agents no longer share. A comparison that trains only the unconstrained baseline and the fully constrained agent cannot separate what the constraint achieved from what the extra network alone

changed.

- **Seed variance.** A single training run under one random seed says little about what a method will do on the next run [6]. Published comparisons of constrained against unconstrained manipulation policies typically report one run, or a mean over few, without a measure of how much the outcome would vary if the same method were retrained.

Section 2.11 sets out this gap in full against the closest published comparison [7]. What matters here is the consequence for design. Chapter 3 describes a comparison built to answer both problems at once. Alongside the unconstrained baseline it places a control arm, identical to the constrained agent in every respect except that its Lagrange multiplier is pinned to zero, so that whatever the cost critic does on its own can be measured apart from whatever the constraint does. The constrained agent is trained at two episodic cost budgets, 25 and 15, because a tighter budget and a looser one make different demands of the policy. Every arm is trained on five independent random seeds, and every reported quantity carries its spread across those seeds instead of standing as a single number.

The control arm turned out to reproduce the unconstrained baseline exactly, at the level of the stored network weights rather than merely to within statistical noise, so the two are reported jointly as a single arm throughout Chapter 4. The isolation this design was built to provide therefore measures the cost critic's independent effect as exactly zero, and Section 4.2 gives that verification in full. Three arms are reported in the end: the joint baseline, and the constrained agent at each of its two budgets.

The work also continues a line of manipulator research from the same laboratory. The preceding study [4] addressed the perception and motion side of manipulation, tracking and following a target with classical image-based visual servoing. This thesis addresses the control policy instead, with reinforcement learning, and adds a safety constraint that a classical visual servoing loop does not itself provide.

1.3 Objectives

The general objective of this thesis is to determine whether expressing a manipulability safety requirement as an explicit constraint, rather than as a term in the reward, changes the safety behaviour of a learned grasping policy on a UR5e manipulator, and whether it changes how predictable that behaviour is across independent training runs.

The specific objectives are as follows:

- To implement an unconstrained PPO baseline and a constrained PPO-Lagrangian agent for a UR5e pick-and-lift grasping task in Isaac Lab, trained under a shared floor on manipulability.

- To isolate the effect of the constraint from the effect of the cost critic that implements it, by training a control arm that carries the cost critic with its Lagrange multiplier held at zero throughout training.
- To establish whether the constrained agent's behaviour depends on how tight its budget is, by training it at two episodic cost budgets, 25 and 15, instead of one.
- To train every arm on five independent random seeds and report task performance and safety outcomes as a distribution across those seeds instead of as a single mean.
- To evaluate every trained policy on frozen, held-out episodes, so that the reported safety and task figures describe the policy that would actually be deployed rather than a still-learning one.
- To determine, from the resulting distributions, whether the constraint changes the average safety outcome of a training run, the spread of that outcome across runs, or both.

1.4 Scope

This thesis was registered under the title *Safe Adaptive Image-Based Visual Servoing with Constrained Reinforcement Learning for Precision Grasping on a UR5 Manipulator: From Simulation to Real Hardware*, and was proposed as a three-layer programme, not as Layer 1 alone. The title on this book is narrower because the delivered work is narrower, and the paragraphs below say exactly where the boundary falls.

Layer 1, delivered here, is safe reinforcement learning for grasping in simulation, using privileged pose information rather than vision. Layer 2 would have closed the loop with vision: an image-based visual servoing scheme in which the image Jacobian relating pixel motion to joint motion is tuned by the learned policy rather than fixed in advance, replacing the privileged pose feedback of Layer 1 with an eye-in-hand RGB-D camera and object detection. It would have been a direct upgrade of the predecessor project in this laboratory, which used a fixed, hand-derived image Jacobian for the same visual-servoing task [4]. Layer 3 would have carried the resulting controller onto the physical UR5e over ROS 2, closing the gap between the Robotiq 2F-85 gripper modelled in simulation and the different gripper fitted to the laboratory's real arm.

Only Layer 1 is delivered. Layers 2 and 3 are not attempted, and that is stated here as a deliberate narrowing of scope, decided against a fixed submission date, rather than as an unmet target. Building an evaluation protocol able to separate a safety constraint's effect from an implementation artefact, and to characterise it across independent seeds, could not be done to a defensible standard alongside two further systems-integration efforts of comparable size. Both dropped layers are carried forward as future work in Chapter 6.

Layer 1 on its own remains a substantial question. Constraining a policy's behaviour with

an explicit safety budget, rather than through a shaped reward, is already active practice on UR-class hardware doing collision avoidance around a moving person [1], and on other manipulator platforms doing constrained grasping specifically [7]. What this thesis contributes sits inside that same practice: a comparison protocol for measuring what the constraint itself does, isolated from an implementation artefact and reported across seeds rather than as a single run, on a platform common enough in both industry and the published literature that the result should say something beyond this one laboratory's copy of it.

Within Layer 1, the scope is:

- **Task.** Single-object pick-and-lift grasping on a simulated UR5e in Isaac Lab, on the weld-grasp environment described in Chapter 3. The gripper is present and driven, but the moment of contact is abstracted rather than modelled as a full physical grasp.
- **Comparison.** Three algorithm arms: an unconstrained PPO baseline and two PPO-Lagrangian variants at different cost budgets, each trained on five independent random seeds.
- **Safety quantity.** A single constrained quantity, a floor on manipulability. Joint-limit proximity and workspace clearance are measured and reported but are not themselves constrained.
- **Evaluation.** The frozen policy produced by each run, assessed on held-out episodes with the multiplier and exploration noise fixed, not the training-time behaviour of a policy still being updated.
- **Algorithms.** The comparison uses established methods, PPO and PPO-Lagrangian; it does not propose a new constrained-RL algorithm. The contribution claimed is the comparison protocol and what it finds, not a new optimiser.
- **Hardware validation.** All results are produced in simulation. No policy from this thesis is run on the physical UR5e. That step belongs to Layer 3, out of scope here and named as future work in Chapter 6 rather than as a claim already tested.

CHAPTER 2

LITERATURE REVIEW

2.1 How to read this chapter

This chapter makes one argument: that a safety requirement on a learned manipulation policy is better expressed as a constraint with a stated budget than as a penalty term inside the reward, and that the published work closest to this thesis leaves two questions about that claim unanswered. Everything here supports, qualifies or positions that argument.

It proceeds in four movements. Sections 2.2 to 2.5 build the machinery: why reward shaping is an awkward vehicle for safety, what a constrained Markov decision process is, and which deep constrained algorithm this thesis uses. Sections 2.6 to 2.8 turn to robotics and set the reviewed studies side by side so their methods can be compared directly. Sections 2.9 and 2.10 treat the two quantities on which the work turns: the manipulability measure that defines its safety constraint, and the seed-to-seed variance that governs how its results must be reported. Sections 2.11 and 2.12 state the gap and close. A reader in a hurry can go straight to Section 2.11, which is what Chapter 1 refers back to and what Chapter 4 answers.

Table 2.1. Section map for this chapter.

§	Subject	Principal sources
2.2	Why safety resists reward shaping	[5], [8]
2.3	Safe RL as a field, and its taxonomy	[9], [10]
2.4	The constrained Markov decision process	[11]
2.5	Deep constrained policy optimisation	[12], [5], [8], [13]
2.6	Safety in robot learning	[14], [15]
2.7	Deep RL on manipulators and on the UR5	[16], [3], [1]
2.8	Experimental methods of the reviewed studies	all nine experimental papers
2.9	Manipulability and singularity avoidance	[17], [18]
2.10	Reproducibility and seed variance	[6]
2.11	Research gap and positioning	[7]

The findings the chapter reaches are collected below, so that the sections that follow read as evidence for them instead of as a survey in search of a conclusion. Each is picked up again in Section 2.12.

Key findings of this chapter

1. A penalty weight and a cost budget are not two spellings of the same instruction. The weight fixes an exchange rate between safety and task progress that the designer must guess in advance; the budget states a limit in the units of the hazard and can be audited after training [5].
2. The constrained Markov decision process is the prevailing formalism for this [10], and its Lagrangian relaxation turns the guessed weight into a dual variable the optimiser sets from the observed violation [11].
3. PPO-Lagrangian is preferred to CPO on published evidence. On the eighteen-environment Safety Gym slate, CPO left a normalised violation of 0.593 against 0.026 for PPO-Lagrangian [8].
4. The Lagrange multiplier under plain dual ascent behaves as an integral controller and oscillates against the cost with a quarter-cycle lag [13]. Any multiplier reported in Chapter 4 is a controller mid-flight, not a converged constant.
5. Yoshikawa's measure $w = \sqrt{\det(JJ^\top)}$ is the volume of the manipulability ellipsoid and the product of the Jacobian's singular values [17]. Prior learning work puts it in the reward with a hand-set weight, in one case $\lambda_3 = 0.05$ [18]. This thesis puts it in a constraint.
6. Random seed alone can produce statistically distinct outcome distributions for one algorithm on one task, at $t = -9.0916$, $p = 0.0016$ [6]. Means over three seeds, which is common practice here, are weak evidence for a safety claim.
7. The closest published comparison of PPO against its constrained variant on a manipulator [7] reports no control arm isolating the cost critic from the constraint, no seed-to-seed dispersion, and no result at more than one budget. These are the three gaps this thesis fills.

2.2 The safety requirement in learned manipulation

A robot manipulator that learns its own control policy poses a problem that one running a hand-written controller does not. A classical controller can be inspected, bounded and argued about before the machine is switched on. A policy obtained by reinforcement learning is the outcome of an optimisation run, and is known only through the statistics of the trajectories it produces. When such a policy is to be deployed on a physical arm sharing its workspace with people, equipment or the workpiece, what it does in the configurations it visits only

rarely is not a secondary concern. It is the condition on which deployment depends at all.

The difficulty is that reinforcement learning offers a single channel through which a designer expresses what the agent should and should not do: the reward function. Safety requirements entered through that channel become penalty terms, and a penalty term needs a weight. That weight is a trade-off rate between task performance and safety, fixed before the designer knows what either will be worth, and it has no natural units. Achiam *et al.* [5] make this argument directly in motivating constrained policy optimisation: for systems interacting physically with or around humans, it is more natural to specify a reward function *and* a set of constraints than to encode both into one scalar objective. Ray *et al.* [8] put it operationally. Too small a penalty and the agent learns unsafe behaviour; too severe a penalty and it may fail to learn at all. A fixed trade-off also cannot respond to the fact that the pressure the reward exerts on the cost changes as the policy improves.

Keeping the requirement outside the reward and imposing it as a constraint with a budget avoids this. A budget can be stated in advance, in the units of the quantity being constrained, and audited afterwards against what the trained policy actually spent. That distinction, that *a shaped reward requires the designer to guess a weight while a constraint requires only that they state a limit*, is the premise of this thesis. Section 2.9 shows it separating two otherwise similar bodies of work on singularity avoidance, where the guessed weight in the nearest such study is a bare 0.05 with no stated derivation.

2.3 Safe reinforcement learning

Safe reinforcement learning studies methods that optimise a policy while respecting some notion of acceptable behaviour during learning, at deployment, or both. The organising survey is that of García and Fernández [9], which splits the literature by where the safety requirement is inserted. One branch modifies the *optimisation criterion*, changing the objective the agent maximises through a risk-sensitive transformation, a worst-case criterion, or explicit constraints. The other modifies the *exploration process*, leaving the objective intact but restricting the actions the agent may try, typically using external knowledge such as a teacher, a demonstration set or a backup controller.

Table 2.2. The two branches of safe reinforcement learning, after [9].

Branch	**Mechanism**	**Evaluated on**
Modify the optimisation criterion	Risk-sensitive or worst-case objectives; explicit constraints on expected cumulative cost	The trained policy, on held-out episodes
Modify the exploration process	Teacher guidance, demonstrations, backup controllers, action shielding	Violations accumulated during training

This thesis belongs to the first branch, and to its constrained-criterion sub-family. The distinction matters for reading Chapter 4: second-branch methods aim to make the learning process itself safe and are evaluated on violations accumulated *during* training, whereas first-branch methods aim to produce a policy that satisfies a requirement once trained, and are properly evaluated on the frozen policy. This work follows the latter convention, and Section 4.1 explains why an earlier version of these results, which read safety figures from training-time telemetry, was withdrawn. The more recent review by Gu *et al.* [10] organises the post-deep-learning field around five open problems, of which the benchmark question is why Chapter 4 reports episodic cost against a declared budget rather than an ad hoc safety score. The same review reports that the CMDP has become the prevailing formalism for safe reinforcement learning, which places this work inside the mainstream of the field and is why the CMDP is adopted here without further argument.

2.4 The constrained Markov decision process

The formal object underlying that branch is the constrained Markov decision process, developed in its modern form by Altman [11]. A standard Markov decision process is the tuple $(\mathscr{S}, \mathscr{A}, P, r, \gamma)$ over a state space, an action space, a transition kernel, a reward function and a discount factor. A CMDP augments this with a cost function that scores each transition for hazard exactly as the reward scores it for progress, and a budget, and it asks for the policy maximising expected return subject to expected accumulated cost staying within that budget. Chapter 3 states the problem formally at Equation (3.1), in the notation of the implementation, and it is not repeated here. That division holds throughout: this chapter carries the equations of the work being reviewed, and Chapter 3 carries those of the method being built.

Two properties matter for the argument. The safety requirement is expressed in the units of the cost rather than of reward, so it can be set from a measurement of the system instead of a tuning exercise, which is what the calibration of Section 3.9 depends on. And the constrained problem admits a Lagrangian relaxation, folding the constraint into the objective through a multiplier that is itself optimised:

$$\max_{\theta} \min_{\lambda \geq 0} \mathscr{L}(\theta, \lambda) = J_r(\pi_\theta) - \lambda \big(J_c(\pi_\theta) - d \big). \tag{2.1}$$

The duality theory licensing that treatment is the contribution of [11], and it is what makes the algorithms of the next section possible: instead of solving a constrained problem directly, they solve a sequence of unconstrained ones in which the penalty weight is driven by the observed violation. That point is easily lost, because a Lagrangian method does end up optimising a weighted sum of reward and cost, which superficially resembles reward shaping. The difference is that the weight is not a hyperparameter. It is a dual variable that rises while

the budget is exceeded and falls once it is met. A useful test is to ask what the designer would change if the hazard became twice as costly to the operator. Under reward shaping they would retune a weight and retrain until the behaviour looked right; under a constraint they would halve the budget and let the optimiser find the weight.

2.5 Constrained policy optimisation in deep reinforcement learning

The baseline algorithm throughout this thesis is proximal policy optimisation [12], which limits the size of each policy update by clipping the probability ratio between the candidate policy and the one that collected the data, so that a large policy change can never be rewarded. Chapter 3 gives the clipped surrogate objective at Equation (3.3) and explains why the clip does the work of a trust region without a constrained optimisation. What matters here is the consequence: PPO attains much of the reliability of trust-region methods at a fraction of their implementation complexity, which is why it is also the algorithm most commonly extended when a constraint is added. Both grasping studies of Section 2.7 use it, at $\varepsilon = 0.2$ in the UR5 case [3], and so does the constrained work nearest this thesis [7].

Constrained policy optimisation (CPO) [5] first brought the CMDP into deep reinforcement learning at scale, replacing the unconstrained trust-region update with

$$\pi_{k+1} = \arg\max_{\pi \in \Pi_\theta} J_r(\pi) \quad \text{s.t.} \quad J_c(\pi) \leq d, \quad D(\pi, \pi_k) \leq \delta, \tag{2.2}$$

solved approximately at every step so that each update both improves the return and maintains near-constraint satisfaction. Its significance here is as much conceptual as algorithmic: it established the constrained formulation as a practical alternative to reward shaping for high-dimensional continuous control, and it supplies the argument quoted in Section 2.2.

The algorithm actually implemented is the Lagrangian variant of PPO, as specified in the Safety Gym benchmark report of Ray *et al.* [8]. Its constrained agents are trained by gradient ascent on θ and descent on λ in Equation (2.1), at a cost limit of $d = 25$ throughout, over 1000-step episodes and three random seeds per environment. That budget of 25 is inherited by Chapter 4 rather than chosen freshly, which removes one degree of freedom from the present design. The second reason to cite that report is empirical, and is why PPO-Lagrangian was preferred to CPO.

CPO retains far more return than either Lagrangian method, but leaves a normalised violation of 0.593, removing only about forty percent of the unconstrained agent's violations where the Lagrangian methods remove roughly ninety-seven percent. Ray *et al.* attribute the shortfall to approximation error in CPO's analytic step, and note that the result contradicts the one reported in [5] on easier environments. Adopting CPO would therefore have required a justification this work does not have.

Table 2.3. Normalised end-of-training metrics on the eighteen-environment Safety Gym slate, three seeds per environment, from [8]. Normalised against unconstrained PPO, so 1.0 is the baseline and lower is safer.

Algorithm	**Return** $\bar{J}_r$	**Violation** $\bar{M}_c$	**Cost rate** $\bar{\rho}_c$
PPO (unconstrained)	1.000	1.000	1.000
TRPO (unconstrained)	1.094	1.132	1.004
PPO-Lagrangian	**0.240**	**0.026**	**0.245**
TRPO-Lagrangian	0.331	0.018	0.265
CPO	0.784	0.593	0.646

Lagrangian methods carry a characteristic weakness, and this thesis meets it directly in Chapter 4. Stooke *et al.* [13] identify the cause precisely: plain dual ascent on λ accumulates the running constraint violation, which is integral control on the constraint and nothing else. An integral controller with no proportional or derivative action overshoots, so the cost and the multiplier oscillate against one another with a quarter-cycle lag, as Figure 2.1 shows.

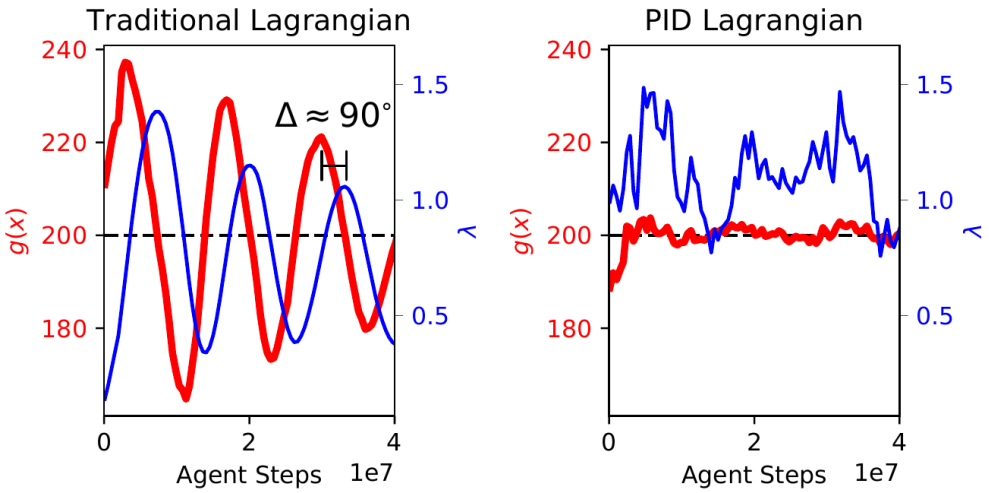

Figure 2.1. Cost $g(x)$ and multiplier λ under the traditional Lagrangian update (left) and under PID control (right). From [13], Fig. 1.

Their remedy treats the multiplier as a control input under full PID control,

$$\Delta_k = J_c(\pi_k) - d, \qquad I_k = \big(I_{k-1} + \Delta_k\big)_+, \qquad \partial_k = \big(J_c(\pi_k) - J_c(\pi_{k-1})\big)_+, \tag{2.3}$$

$$\lambda_k = \big(K_P\Delta_k + K_I I_k + K_D\partial_k\big)_+, \tag{2.4}$$

where $(\cdot)_+$ is projection onto the non-negative reals and $K_P = K_D = 0$ recovers the traditional method exactly. Two implementation details are inherited by this thesis. They maintain a *separate* cost-value approximator alongside the reward-value approximator instead of folding $r + \lambda c$ into a single critic, on the grounds that λ may change quickly, and they rescale the policy gradient by $1/(1+\lambda)$ so that a large multiplier does not enlarge the effective step

size. The first of those is why the constrained agent here carries a network its baseline does not, which is in turn why a control arm is needed at all; Section 2.11 returns to it. Their analysis is used in Section 4.6 to read the multiplier values observed at the end of training, and the PID form is identified in Chapter 6 as the natural extension of this work. It is worth noting that [13] names its PPO-based agent CPPO, the same abbreviation this thesis uses. The benchmark suites have since been superseded: Safety Gymnasium [19] replaces the original environments and ships a library of safe algorithms under a common protocol, and its reporting conventions, episodic cost against a stated budget and violation rates measured per episode, are those adopted in Chapter 4.

2.6 Safety in robot learning

The literature reviewed so far approaches safety from the machine-learning side. Robotics approaches it from the control side, with a vocabulary of invariant sets, barrier certificates and stability guarantees that developed independently. Brunke *et al.* [14] reconcile the two into a common framework spanning learning-based control through to safe reinforcement learning. That reconciliation is needed here because this thesis sits across the boundary: the quantities it constrains (the manipulability measure, the distance to a joint limit, the clearance above the work surface) are control-theoretic constructs describing the kinematic state of the arm, while the mechanism enforcing them is a cost critic and a dual variable.

The survey also draws a distinction that limits what may be claimed here. Methods enforcing a constraint in expectation, as a Lagrangian method does, bound *expected* exposure to unsafe configurations; they do not bound the severity of any individual excursion. Methods supplying a safety certificate, such as a control barrier function filtering the commanded action, offer the latter guarantee but require a model. Section 4.5 reports a counter-result of exactly this shape, and Chapter 6 argues the two mechanisms address different halves of the problem. Within manufacturing specifically, Elguea-Aguinaco *et al.* [15] review reinforcement learning for contact-rich manipulation from 2017 onward, and their survey is used in Chapter 3 to establish which design choices made here follow established practice.

2.7 Deep reinforcement learning for robotic manipulation

Applications of deep reinforcement learning to manipulation divide broadly by algorithm family. Shahid *et al.* [16] compare PPO against soft actor-critic on a grasping task with a Franka Emika Panda in MuJoCo, splitting the task into a reaching and a lifting phase with a separate dense reward for each and adding penalties on joint-position proximity, joint acceleration and obstacle approach by fine-tuning rather than all at once. The policy runs at 250 Hz against a 500 Hz joint controller and trains for 10^7 steps. Evaluated on the physical Panda with no real-world training data, it succeeds ten times out of ten on each of four objects.

That study is the methodological template for an on-policy against off-policy comparison, it demonstrates zero-shot transfer to hardware, and it is the precedent for the off-policy arm planned for this study and ultimately not trained (Section 4.9).

Work specific to the UR5 is less common. Ferreira and Barbosa [3] train a UR5 with a Robotiq 2F-85 gripper to grasp randomly positioned objects in PyBullet at 240 Hz, comparing soft actor-critic against PPO under identical conditions with domain randomisation. Their scene, in Figure 2.2, is the closest published analogue to the environment of this thesis in both robot and gripper.

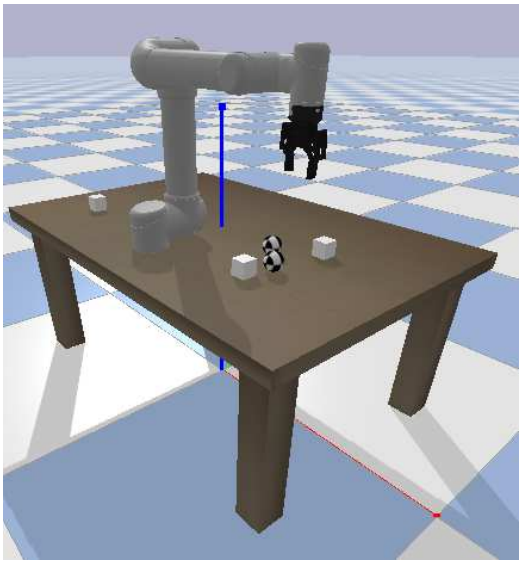

Figure 2.2. A UR5 with a Robotiq 2F-85 gripper in PyBullet. From [3], Fig. 1, CC BY 4.0.

Their reward is shaped,

$$R(s,a) = -\lambda \, \| p_g - p_o \|_2 + r_s \, \mathbb{I}_{\text{grasp}} + r_p \, \mathbb{I}_{\text{pose}}, \tag{2.5}$$

with $\lambda = 0.1$ weighting the gripper-to-object distance penalty, $r_s = 10$ for a grasp held at least 0.5 s and $r_p = 5$ for correct placement orientation. Their own ablation shows how much rests on those hand-set numbers: removing the distance term costs ten percentage points of success, the grasp reward twelve, the pose reward eight, and reducing the function to its two sparse indicators eight. Both agents trained for 100,000 timesteps at a learning rate of 3×10^{-4}, batch size 64, two-layer 256-unit MLP. They report grasp success of 87 % for SAC and 82 % for PPO, with post-grasp placement success of 75 % and 68 %. Those are the closest published figures for a learned UR5 grasping policy, with one qualification that must be carried into any comparison: their task requires a full physical grasp in which the fingers close and the contact must be stable, whereas this thesis abstracts the grasp as a weld for the reasons given in Section 3.5. The success rates of Chapter 4 are therefore not directly comparable to theirs and are not presented as though they were.

On the safety side, Xia *et al.* [1] apply deep reinforcement learning to proactive obstacle avoidance for a UR5e in a human-robot collaborative assembly cell, using an RGB-D camera and a wrist force-torque sensor with a soft actor-critic agent under a composite reward, and report a 93 % success rate in avoiding dynamic obstacles while still reaching the goal pose. Their work establishes that safe operation of this specific manipulator is an active industrial concern and not an academic construction, which is the premise on which the practical relevance of this thesis rests. It also illustrates the alternative design route: where this thesis expresses safety as a budget on a kinematic cost, they express it as another term inside a composite reward, and 93 % is a success rate rather than a bound on anything.

2.8 Experimental methods of the reviewed studies

Describing each study in turn makes it hard to see what the field holds fixed and what it varies. Table 2.4 therefore sets the experimental apparatus side by side. It covers only the papers reporting their own experiments; the three surveys are excluded.

Table 2.4. Experimental apparatus of the reviewed studies. A dash means the paper does not state a seed count.

Study	Platform	Simulator	Algorithms	Seeds	Headline result
Ray *et al.* [8]	Point, Car, Doggo	MuJoCo / Safety Gym	PPO, TRPO, both Lagrangian, CPO	3	PPO-Lag. violation 0.026 vs CPO 0.593 (normalised)
Stooke *et al.* [13]	Doggo	Safety Gym	CPPO with PID multiplier	–	Cost oscillation damped; violations reduced
Henderson *et al.* [6]	MuJoCo locomotion	OpenAI Gym	TRPO, PPO, DDPG, ACKTR	5 (and 10)	Seed alone gives distinct distributions, $p = 0.0016$
Shahid *et al.* [16]	Franka Panda	MuJoCo	PPO, SAC	–	100 % grasp success, zero-shot on hardware
Ferreira and Barbosa [3]	UR5 + 2F-85	PyBullet	PPO, SAC	–	SAC 87 % vs PPO 82 % grasp success
Xia *et al.* [1]	UR5e	Custom + ROS	SAC	–	93 % dynamic obstacle avoidance
Shen *et al.* [18]	4-DOF planar	Custom	SAC in Jacobian null space	5	96.8 % vs 77.4 % avoidance; higher mean w
Khan *et al.* [7]	Franka Panda	Safety Gym	PPO, cPPO	–	cPPO cost 12.32 vs PPO 17.76 (SOPM)
This thesis	**UR5e**	**Isaac Sim**	**PPO (baseline), cPPO $d = 25$, cPPO $d = 15$**	**5**	**Chapter 4**

Three patterns in that table shape decisions made in Chapter 3. The cost limit $d = 25$ re-

curs: Ray *et al.* set it as the Safety Gym reference configuration [8], Khan *et al.* inherit it unchanged along with $\gamma = 0.99$, $\lambda_{\mathrm{GAE}} = 0.97$, a penalty learning rate of 5×10^{-2} and a penalty initialised at 1.0 [7], and this thesis inherits it in turn. A number reused across three studies is not thereby correct, but it does mean the budget in Chapter 4 is a convention and not a free parameter tuned to flatter the result. Network capacity is also uniformly small, two hidden layers of 64 to 256 units across all of them, and none of the reported outcomes appears limited by it, which is worth knowing before attributing any difference in this thesis to architecture.

The third pattern is one of absence, and it matters most. Of the eight experimental studies, three report a seed count and five do not. None of the three reports per-seed dispersion; they report a mean, or a shaded band whose composition is not decomposed. Section 2.10 takes up what follows.

2.9 Manipulability and singularity avoidance

The constraint this thesis is built around is a floor on manipulability. One qualification belongs here rather than later, because it changes how this section should be read. The cost function of Chapter 3 scores three hazards rather than one, and joint-limit proximity is genuinely active alongside manipulability, with the collision term inactive throughout. Across the five unconstrained baseline runs the manipulability term carries about 78 % of the realised episodic cost on average and joint-limit proximity about 22 %, but that average is unstable: three of the five runs spend their whole budget on near-singular configurations and never approach a joint limit, while one spends most of its budget on joint-limit proximity instead (Section 3.9). Manipulability is therefore the framing constraint, the one that motivates the design, and on the mean the dominant one, but it is not the only term the budget is spent on and it does not dominate on every seed. This section develops the measure carefully because the argument of the thesis rests on it and because the difference between this work and its nearest neighbour is entirely a difference in where that quantity is placed.

2.9.1 The measure and what it means

Consider a manipulator with n joints, joint vector $\theta \in \mathbb{R}^n$, whose task is described by $m \leq n$ manipulation variables $\mathbf{r} \in \mathbb{R}^m$. Differentiating the forward kinematics gives

$$\dot{\mathbf{r}} = J(\theta)\,\dot{\theta}, \qquad J(\theta) \in \mathbb{R}^{m \times n}, \tag{2.6}$$

with J the manipulator Jacobian. Yoshikawa [17] defines a configuration θ^* to be *singular* when $\operatorname{rank} J(\theta^*) < m$. At such a configuration the end-effector cannot be moved instantaneously along at least one Cartesian direction, whatever joint velocities are commanded. As

a scalar index of distance from that condition he proposes

$$w(\theta) = \sqrt{\det\big(J(\theta)J(\theta)^{\top}\big)}. \tag{2.7}$$

Three properties established in [17] explain why this particular combination is the right one, and each is used somewhere in this thesis. First, writing the singular value decomposition $J = U\Sigma V^{\top}$ with singular values $\sigma_1 \geq \cdots \geq \sigma_m \geq 0$, the measure is their product,

$$w = \sigma_1 \sigma_2 \cdots \sigma_m, \tag{2.8}$$

so it collapses as soon as any single σ_i does. It is sensitive to the loss of *one* direction of motion even when the arm remains perfectly capable in every other direction, which is exactly the failure that matters. Second, the set of end-effector velocities reachable from unit-norm joint velocities is an ellipsoid with principal axes $\sigma_1 u_1, \ldots, \sigma_m u_m$, and w is proportional to its volume, so the index measures how much end-effector motion a unit of joint effort buys. Third, when $m = n$ Equation (2.7) reduces to $w = |\det J|$; the UR5e has six joints controlling a six-dimensional pose, so this is the form that applies here.

Yoshikawa's simplest worked example makes the quantity concrete. For a planar two-link arm with link lengths ℓ_1, ℓ_2 and joint angles θ_1, θ_2,

$$w = \ell_1 \ell_2 |\sin \theta_2|, \tag{2.9}$$

maximised at $\theta_2 = \pm 90^{\circ}$ and vanishing when the arm is fully extended or fully folded, as Figure 2.3 shows. He notes that a human arm, treated as a two-link mechanism, satisfies $\ell_1 \approx \ell_2$ and is habitually used with the elbow near a right angle, so people appear to adopt high-manipulability postures without being told to. The intuition to keep is that a straightened arm is a weak arm: it can still push along its own length, but it can barely move sideways, and a controller asking it to do so will demand very large joint rates.

That consequence is why a floor on w counts as a safety constraint rather than a performance target. Near a singularity the inverse mapping is ill-conditioned, so a small commanded Cartesian velocity maps to a large joint velocity. On a real UR5e whose joints are capped at $180^{\circ}\mathrm{s}^{-1}$, that appears either as a violent motion or as a protective stop. Yoshikawa anticipates this by normalising the Jacobian by the per-joint maximum velocities when those differ, which for the UR5e they do not, since all six joints share the same ceiling. The unnormalised form is therefore used directly in Chapter 3, where the cost term is an excursion below a floor $w_{\min}$, and where Section 3.9 records the recalibration of that floor from 0.045 to 0.06 after the baseline policy's natural operating distribution was measured.

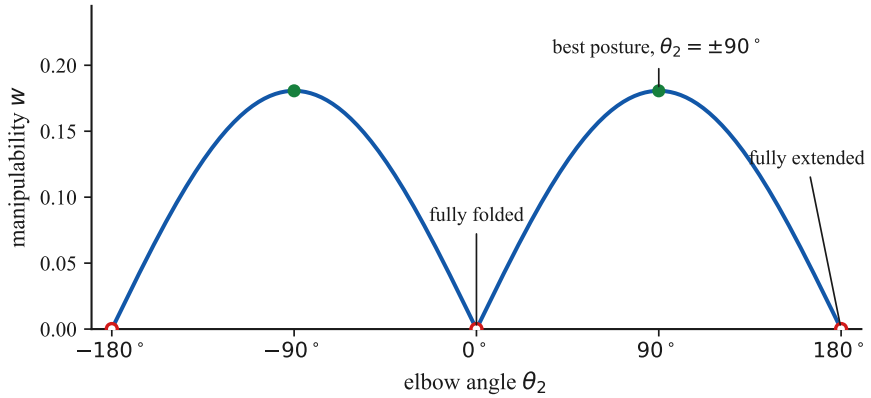

Figure 2.3. Manipulability of a planar two-link arm with equal links, from Equation (2.9). It falls to zero at both the folded and the extended posture and peaks at a right angle.

2.9.2 *The same measure, placed differently*

Shen *et al.* [18] address reactive obstacle avoidance for redundant manipulators by learning self-motion in the null space of the Jacobian, so the commanded task velocity is preserved exactly while the learned action redistributes the remaining freedom:

$$\dot{q}_N = \left(I - J^{\dagger}J\right)\phi, \qquad \dot{q}_D = J^{\dagger}\dot{x}_D, \tag{2.10}$$

with ϕ the learned action and $J^{\dagger}$ the pseudo-inverse. Their agent is soft actor-critic and their reward is the weighted sum

$$r = \lambda_1 r_o + \lambda_2 r_a + \lambda_3 r_m, \qquad r_o = \sum_{i=1}^{n} \min\left(\frac{\|d_i\|}{d_s} - 1, 0\right), \qquad r_m = \sqrt{\det\left(JJ^{\top}\right)}, \tag{2.11}$$

in which r_o penalises obstacle proximity inside a safe distance d_s, r_a penalises null-space joint motion, and r_m is Yoshikawa's measure entered directly as a reward term. Their reported weights are $\lambda_1 = 1$, $\lambda_2 = 0.2$ and $\lambda_3 = 0.05$, with a flat penalty of -10 for a collision or joint-limit breach. The method works: over 1000 randomly generated obstacle configurations it raises avoidance success from 77.4 % to 96.8 % against a gradient-projection baseline in the two-obstacle scenario while holding a higher mean manipulability throughout (3.72 against 3.63), and it computes the null-space motion faster because a forward pass replaces an optimisation. Figure 2.4 shows what that looks like in configuration space.

The objective pursued there is the same as the one pursued here, and the quantity is identical down to the square root. The mechanism is not. Placing manipulability in the reward requires a weight relating a unit of w to a unit of task progress, and $\lambda_3 = 0.05$ is that weight. Nothing in the paper derives it, and nothing could, because there is no physical exchange rate

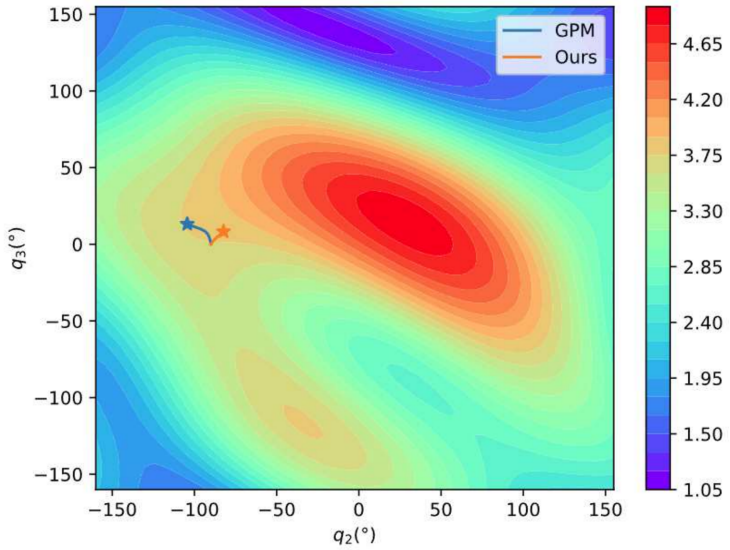

Figure 2.4. Manipulability over the q_2–q_3 plane, with the paths taken by the baseline and by the learned policy from a common start. From [18], Fig. 12, CC BY 4.0.

between an ellipsoid volume and a metre of end-effector travel. It is a number that produced acceptable behaviour. Placing the same quantity in a constraint requires instead a budget, a statement of how much cumulative proximity to singularity an episode may incur, which can be measured from a baseline policy as Section 3.9 does and audited against the trained one as Section 4.5 does. Both approaches seek the same behaviour from the same measure. One obtains it by tuning a trade-off; the other by declaring a limit.

2.10 Reproducibility and seed variance

A final strand of the literature is methodological instead of algorithmic, and it bears on the central experimental finding of this work. Henderson *et al.* [6] examine reproducibility in deep reinforcement learning and identify several sources of variation that are not the contribution being claimed: network architecture and activation function, reward scaling, choice of environment, the baseline codebase used, and the random seed. Their treatment of the last is unusually direct. They run ten trials of TRPO on HalfCheetah-v1 under one fixed hyperparameter configuration, varying nothing but the seed, split those ten arbitrarily into two groups of five, and average each group. If seed were noise around a stable mean the two curves would coincide. Figure 2.5 shows that they do not. A two-sample t-test across the training distribution gives $t = -9.0916$ and $p = 0.0016$: two groups drawn from the same algorithm, the same task and the same settings belong to statistically distinct distributions.

Their bootstrap intervals give the size of the effect. On HalfCheetah-v1 the 95 % interval for DDPG runs from 3664 to 6574 about a mean of 5037, a span of more than half the mean, so two papers reporting single runs of the same algorithm on that environment could differ by a

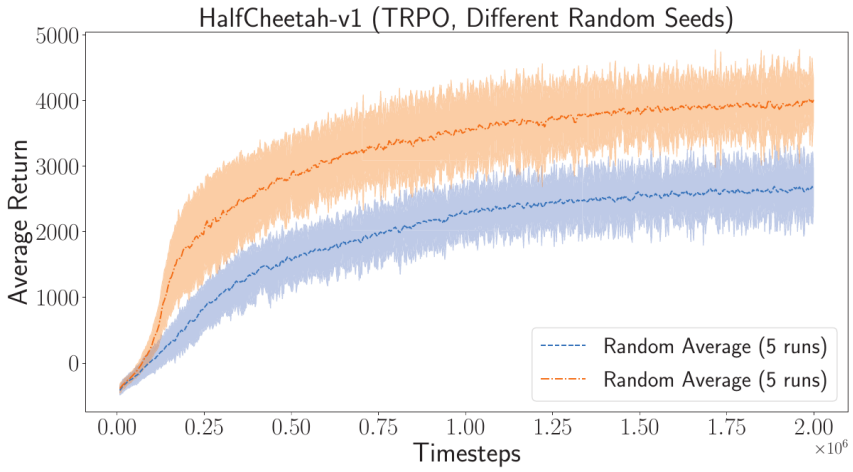

Figure 2.5. TRPO on HalfCheetah-v1, two groups of five random seeds under identical settings ($t = -9.0916$, $p = 0.0016$). From [6], Fig. 5.

factor approaching two without either having done anything wrong. Their recommendation follows: report over multiple independent seeds with a measure of dispersion, treat comparisons with corresponding caution, and do not select the best of several runs. They decline to name a required number of trials, but note that averaging fewer than five is common in published work and is not defensible.

Set that against Table 2.4. Ray *et al.* use three seeds [8]; Shen *et al.* use five and plot the minimum-to-maximum band [18]; five of the eight experimental studies reviewed here, including the one nearest this thesis, do not state a seed count at all. That is not a criticism of any individual paper, since the convention is field-wide, but it is an observation about what the published safety numbers can support.

Three consequences follow, of different kinds. Procedurally, the protocol of Chapter 3 trains every arm on five independent seeds (1, 3, 4, 52 and 54) and reports dispersion across them rather than a single figure, justified by reference to [6] rather than by assertion, which is why this section sits in the literature review and not in the methodology. Interpretively, the seed-to-seed spread reported in Section 4.6 is an instance of a documented property of the method and is presented there as a measured quantity and not as a surprising discovery. And there is a consequence for what a safety claim is for. Henderson *et al.* are concerned with fair comparison between algorithms, which is a concern of authors. An integrator deploying a policy has a different one: they do not obtain the mean over training runs they will never perform, but the single policy they happened to train, on the single seed they happened to use. For that reader the dispersion is not an error bar around the result. It is the result. That reframing is what motivates the question Chapter 4 actually asks, which is whether a constraint tightens the spread of safety outcomes as well as shifting their centre.

2.11 Research gap and positioning

The nearest published analogue to this study is that of Khan *et al.* [7], which compares a model-free PPO baseline against its constrained variant for precision grasping on a Franka Panda in Safety Gym, and introduces a tactile feedback scheme to accelerate learning under safety compliance. Their agent is a two-layer 64-unit MLP actor-critic on an eight-dimensional observation, trained for 250 epochs of 2000 steps at the Safety Gym reference settings, including the cost limit of 25 this thesis also uses. On single-object precision manipulation they report a final average episodic cost of 12.32 for cPPO against 17.76 for PPO, and on the harder multi-object task 29.95 against 38.80, in both cases giving up a little reward for a substantially lower cost. It is the closest match in method, framing and intent, and this thesis continues that line of work rather than departing from it.

Three gaps are nonetheless visible in that line. Table 2.5 states them in the form they take as design requirements, together with what this thesis does about each.

Table 2.5. The three gaps and the design response to each.

	What the literature does	**Why it is a problem**	**What this thesis does**
Gap 1. No control arm	Trains cPPO against PPO and reports the difference [7]	cPPO also adds a cost-value network sharing an optimiser, buffer and RNG stream with the policy, so the difference sums an implementation effect and a constraint effect	Trains a control arm carrying the cost critic with λ pinned to zero, and measures the implementation term directly (Section 4.2)
Gap 2. No dispersion	Reports means; five of eight studies do not state a seed count	A practitioner deploys one policy, not a mean over runs they will never perform [6]	Trains five seeds per arm (1, 3, 4, 52, 54) and reports the spread of every quantity (Section 4.6)
Gap 3. One budget only	Fixes $d = 25$ and never varies it [8, 7]	A single budget cannot show whether the effect is a property of the constraint or an artifact of one threshold	Trains a second constrained arm at $d = 15$, so the effect can be read against how hard the constraint is applied (Section 3.10.1)

Gap 1 is not a hypothetical concern. An earlier version of the present work reported a large advantage for the constrained agent that was later traced to exactly this class of implementation artifact and withdrawn in full, as Chapter 4 records. Written as an identity, the difference prior work reports splits into two terms of different kinds:

$$\underbrace{Q(\text{cPPO}) - Q(\text{PPO})}_{\text{what is reported}} = \underbrace{\big[Q(\text{ctrl}) - Q(\text{PPO})\big]}_{\text{implementation term}} + \underbrace{\big[Q(\text{cPPO}) - Q(\text{ctrl})\big]}_{\text{constraint term}}, \tag{2.12}$$

where *ctrl* carries the cost critic with its multiplier pinned to zero. The design that resolves all three gaps is shown in Figure 2.6. One detail of it is worth stating in advance, because it is what makes Equation (2.12) usable rather than merely true. A plain PPO arm was trained alongside the control arm on the same seeds, and its stored network weights proved byte-identical to the control arm's. The implementation term is therefore measured at exactly zero rather than estimated, the control arm stands in for plain PPO throughout, and the arm labelled "PPO (baseline)" in Chapter 4 is that control arm. Section 4.2 gives the verification.

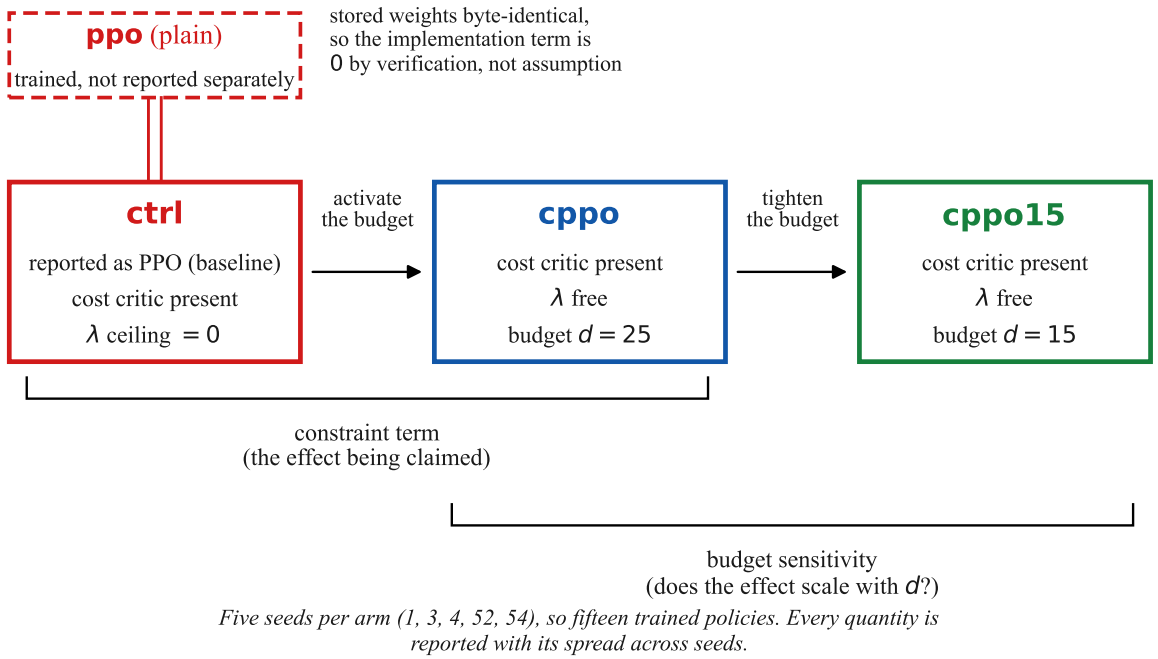

Figure 2.6. The three trained arms, and the two comparisons they support.

Table 2.6 sets that design against the two studies it most closely resembles.

The work is also positioned locally. It continues a line of manipulator research in the same laboratory, of which the immediately preceding study [4] implemented classical image-based visual servoing with CSRT tracking on a five-degree-of-freedom manipulator. Where that work addressed the perception and servoing loop with classical methods, the present work addresses the control policy with learned ones, and adds the safety constraint that classical visual servoing does not itself provide.

2.12 Summary

The path from the first section of this chapter to the last is short enough to retrace in one pass, and doing so makes clear why the experiment of Chapter 4 takes the form it does.

It begins with a mismatch. Reinforcement learning gives a designer one channel through which to say what the agent should do, and safety requirements do not fit down it comfortably. A hazard entered into the reward becomes a penalty, a penalty needs a weight, and that

Table 2.6. Positioning against the two nearest published studies.

Property	**Khan *et al.*** [7]	**Shen *et al.*** [18]	**This thesis**
Manipulator platform	Franka Panda	4-DOF planar	UR5e
Safety expressed as	Constraint	Reward term	Constraint
Safety quantity	Collision cost	Manipulability w	Manipulability w, joint limit, collision
Constrained algorithm	PPO-Lagrangian	SAC (unconstrained)	PPO-Lagrangian
Cost-critic control arm	×	×	✓
Seed count stated	×	✓ (5)	✓ (5)
Per-seed dispersion reported	×	×	✓
Budget varied	× ($d = 25$ only)	n/a	✓ ($d = 25$ and 15)
Frozen-policy evaluation	×	✓	✓

weight is an exchange rate between injury and task progress that nobody can derive [5, 8]. Section 2.9 produced the concrete instance: in the closest published treatment of singularity avoidance by learning, that exchange rate is $\lambda_3 = 0.05$, and the paper offers no account of where it came from [18]. That is not a lapse on the authors' part, since there is nothing available to derive it from.

The constrained Markov decision process resolves the mismatch by moving the requirement out of the objective and into a separate condition with a budget [11], an approach that Gu *et al.* [10] report has become the prevailing formalism. Its Lagrangian relaxation does eventually optimise a weighted sum, but the weight is now a dual variable driven by the measured violation. Among the deep algorithms implementing this, PPO-Lagrangian was chosen over CPO on published evidence: on the Safety Gym slate CPO removed about forty percent of the unconstrained agent's violations where the Lagrangian methods removed about ninety-seven [8]. That choice carries a known cost, which Stooke *et al.* [13] diagnose exactly. Plain dual ascent is integral control, integral control overshoots, and the multiplier and the cost consequently chase one another around the budget. Any λ reported in this thesis is a controller mid-flight, and the separate cost critic the same authors recommend is what later forces the control arm into the experimental design.

Turning to robotics narrowed the field quickly. Learned grasping on manipulators is well established and the numbers are respectable: 100 % grasp success with zero-shot hardware transfer on a Franka Panda [16], 87 % for SAC against 82 % for PPO on a UR5 with a Robotiq gripper [3], and 93 % dynamic obstacle avoidance on a UR5e in a human-robot cell [1]. None of them is a bound, however. They are success rates obtained under a shaped reward, and a success rate says nothing about the configurations the arm passed through

on the way. Brunke *et al.* [14] supply the vocabulary joining the safety and manipulation literatures, along with the caveat this thesis must respect: a constraint in expectation bounds average exposure, not the severity of any one excursion. The operative quantity itself came from an older source. Yoshikawa's measure $w = \sqrt{\det(JJ^\top)}$ is the product of the Jacobian's singular values and the volume of the velocity ellipsoid [17], which is why it collapses when the arm loses a single direction of motion instead of degrading gradually, and why a modest Cartesian command near that condition becomes a large joint rate on a real UR5e.

The last piece was methodological, and it changed the question instead of the method. Henderson *et al.* [6] showed that ten runs of one algorithm on one task, differing only in seed, split into two groups of five that are statistically distinct at $p = 0.0016$. Reading Table 2.4 in that light turned an incidental observation into a finding: of the eight experimental studies reviewed, five do not state a seed count and none reports per-seed values, so the published safety numbers in this area are means of unknown composition. Since a practitioner deploys one policy and not a mean, the useful property of a safety mechanism may be how tightly it constrains the spread of outcomes rather than where it puts their centre.

Those threads converge on the three gaps of Section 2.11, all visible in the study nearest this one [7]: no control arm separating the effect of adding a cost critic from the effect of activating the constraint, no dispersion behind the reported mean, and no result at any budget other than the inherited $d = 25$. None is a fault of that paper, since all three are field-wide conventions, and all three are answerable with experimental design and no new algorithm.

That is the scope this thesis takes on, and its boundaries should be stated as plainly as its content. It proposes no new optimiser: the constrained agent is PPO-Lagrangian as published [8], deliberately unmodified, because a modified algorithm would reintroduce the confound the control arm exists to remove. It works in simulation, so its claims concern a simulated UR5e and are transferable only in the sense that [16] demonstrates such transfer is possible. It abstracts the grasp as a weld, so its success rates are not comparable with the full-grasp figures of [3]. And although manipulability is the constraint the argument is built on, Section 3.9 shows joint-limit proximity active alongside it, with which of the two consuming the budget varying from seed to seed, so the safety result is not a manipulability result alone.

Within those boundaries it contributes fifteen trained policies across three arms and five seeds: a control arm whose weights are verified identical to plain PPO, so the implementation term is measured at zero rather than assumed away; a constrained arm at the inherited budget; and a second constrained arm at a tighter one, so the effect can be read against how hard the constraint is applied. Every quantity is reported with its spread across seeds. Chapter 3 sets out the method by which that is done.

CHAPTER 3

RESEARCH METHODOLOGY

This chapter is ordered so that each part is defined before it is used. Sections 3.1 to 3.3 describe the simulator, the robot and the vocabulary of reinforcement learning. Section 3.4 states the problem formally, Sections 3.5 to 3.7 build the task, the software and the safety cost that instantiate it, and Section 3.8 develops the algorithm that solves it. The remaining sections cover calibration and the training and evaluation protocols. The opening sections are deliberately descriptive: the argument for expressing a safety requirement as a constraint rather than as a reward term, and the published evidence behind the algorithm chosen, belong to Chapter 2 and are not restated here.

3.1 Simulation environment: Isaac Sim and Isaac Lab

NVIDIA Isaac Sim is a robotics simulator built on the Omniverse platform. Scenes are described in Universal Scene Description (USD) files, which carry a robot's link geometry, joint definitions, mass and inertia properties and collision meshes in one asset, and the physics is solved by PhysX. Rigid-body dynamics, contact resolution and articulation solving all execute on the GPU, so many independent copies of a scene advance together without their state returning to the CPU between steps. The asset that is simulated is also the asset that is rendered, which is how the gripper fault of Section 3.5 was diagnosed.

Isaac Lab is the reinforcement-learning framework layered on top. It is manager-based: an environment is assembled by declaring configuration objects for observations, actions, rewards, terminations, events and commands, rather than by writing a monolithic step function. A task defined this way is retargeted onto different hardware by replacing the robot articulation, the action term and the end-effector frame, leaving the reward and termination structure of a tested environment untouched. Isaac Lab has no publication of its own; its predecessor Orbit is the citable reference [20], and the GPU-resident training design both inherit was introduced with Isaac Gym [21].

Table 3.1 gives the software stack. Two of the versions are load-bearing rather than incidental: the RTX 5090 is a Blackwell GPU (compute capability sm_120), which is not supported by PyTorch builds before 2.7 or by NVIDIA drivers before the 570 series, so the CUDA 12.8 wheels and the 580-series driver are requirements.

The Isaac Lab version is held at exactly 2.3.0 rather than at the 2.3 release line. The line advanced to 2.3.1 during this work, and 2.3.1 requires a URDF importer version that the

Table 3.1. Software stack, as pinned for every run reported in this thesis.

Component	Version
Simulator	NVIDIA Isaac Sim 5.0.0
RL framework	Isaac Lab 2.3.0
On-policy trainer	`rsl_rl` 3.0.1 (installed by Isaac Lab)
Alternate RL library	`skrl` 2.1.0 (used only for the off-policy smoke test)
Python	3.11, conda environment `isaaclab`
Deep-learning framework	PyTorch 2.7.0+cu128
CUDA toolkit	12.8
GPU	NVIDIA RTX 5090, Blackwell, sm_120
NVIDIA driver	580 series
Host	Intel i9, 64 GB RAM, Ubuntu

installed Isaac Sim does not ship, which stopped training at start-up. Every run reported in Chapter 4 was produced from one frozen version of the code, with no change to the environment, the algorithm or the configuration between arms.

Key points

- Isaac Sim solves the physics on the GPU and Isaac Lab declares the task through managers, which is what makes thousands of parallel environments and a hardware retarget practical.
- The stack is pinned, not merely recorded. PyTorch 2.7 with CUDA 12.8 and a 580-series driver are requirements of the Blackwell GPU, not preferences.
- Isaac Lab is held at 2.3.0 exactly. The 2.3 line moved to 2.3.1 mid-project and broke start-up through a URDF importer version it requires.

3.2 The UR5e manipulator

The platform is a Universal Robots UR5e, a six-degree-of-freedom collaborative arm. Table 3.2 gives its published specifications [2] against the modelling decision each one drives, because several numbers appearing later in this chapter are datasheet limits carried into the simulator rather than values chosen for training convenience.

The last two rows carry a distinction that the rest of the chapter depends on. A real UR5e already has a certified safety controller. Nothing in this thesis replaces it, competes with it, or is evaluated against it. The simulation borrows the arm's physical envelope, its reach and its speed ceiling, and tests whether a learned policy can be held inside a kinematic safety budget by the optimiser itself, which is a different question from whether a hardware interlock can stop the arm.

Table 3.2. UR5e specifications and where each one enters the method.

Property	Value	Where it is used here
Degrees of freedom	6 revolute	Six-dimensional action space (Section 3.5.1)
Payload	5 kg	Not exercised; the grasp is a weld (Section 3.5)
Reach	850 mm	Bounds the goal-sampling box, far corner 0.84 m
Repeatability	$\pm$0.03 mm	Sets the scale at which a 1 cm goal tolerance is loose
Mass	20.6 kg	Not exercised; the arm is fixed-base in simulation
Maximum joint speed	180 deg/s (π rad/s)	Velocity limit in the simulated articulation
Safety functions	17, configurable	**Not used.** The learned constraint is what is under test
Certification	ISO 13849-1 Cat. 3 PL d; ISO 10218-1	Context only; no certified function is simulated

The six revolute joints are named in Table 3.3, with the working range of each and the home configuration the environment resets to. The joint-limit cost term of Section 3.7 measures clearance against these ranges, so the naming is used again there and in the per-joint reporting of Chapter 4.

Table 3.3. The six arm joints, their working range and the reset configuration used in this work.

Joint	Working range	Home (rad)	Home (deg)
`shoulder_pan`	$\pm$360°	0.00	0.0
`shoulder_lift`	$\pm$360°	-1.20	-68.8
`elbow`	$\pm$360°	1.40	80.2
`wrist_1`	$\pm$360°	-1.75	-100.3
`wrist_2`	$\pm$360°	-1.57	-90.0
`wrist_3`	$\pm$360°	0.00	0.0

The home pose places the arm reaching forward over the table with the wrist pointing down, which is the configuration every episode starts from before the cube and goal are randomised.

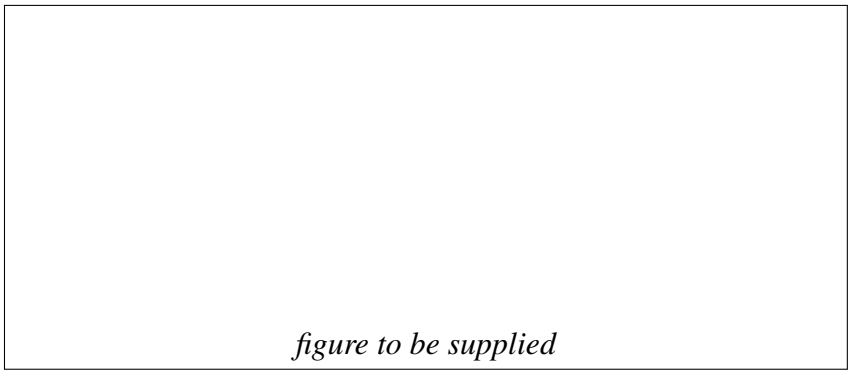

Figure 3.1. The UR5e in the simulated workspace.

3.3 Reinforcement learning, on-policy and off-policy

A reinforcement-learning agent observes the state of its environment, selects an action from a policy, receives a scalar reward, and lands in a new state. Repeating this produces a trajectory, and the object of training is a policy that maximises the reward accumulated along trajectories it generates. In this work the policy is a neural network that maps the observation vector of Section 3.5.1 to a distribution over six joint-position targets, and training adjusts the network weights.

Algorithms of this kind are organised by two questions: whether the agent has a model of the environment, and what it learns. Figure 3.2 shows the taxonomy. A model-based agent has, or learns, a function predicting state transitions and rewards, which lets it plan ahead, but a learned model is exploitable: an agent can find a policy that scores well against the model's errors and behaves badly in the real environment. This work is model-free, which is where the great majority of published manipulation results sit. Within model-free methods the split is between policy optimisation and Q-learning, with a group of algorithms interpolating between them, and that split lines up almost exactly with the on-policy and off-policy division described next.

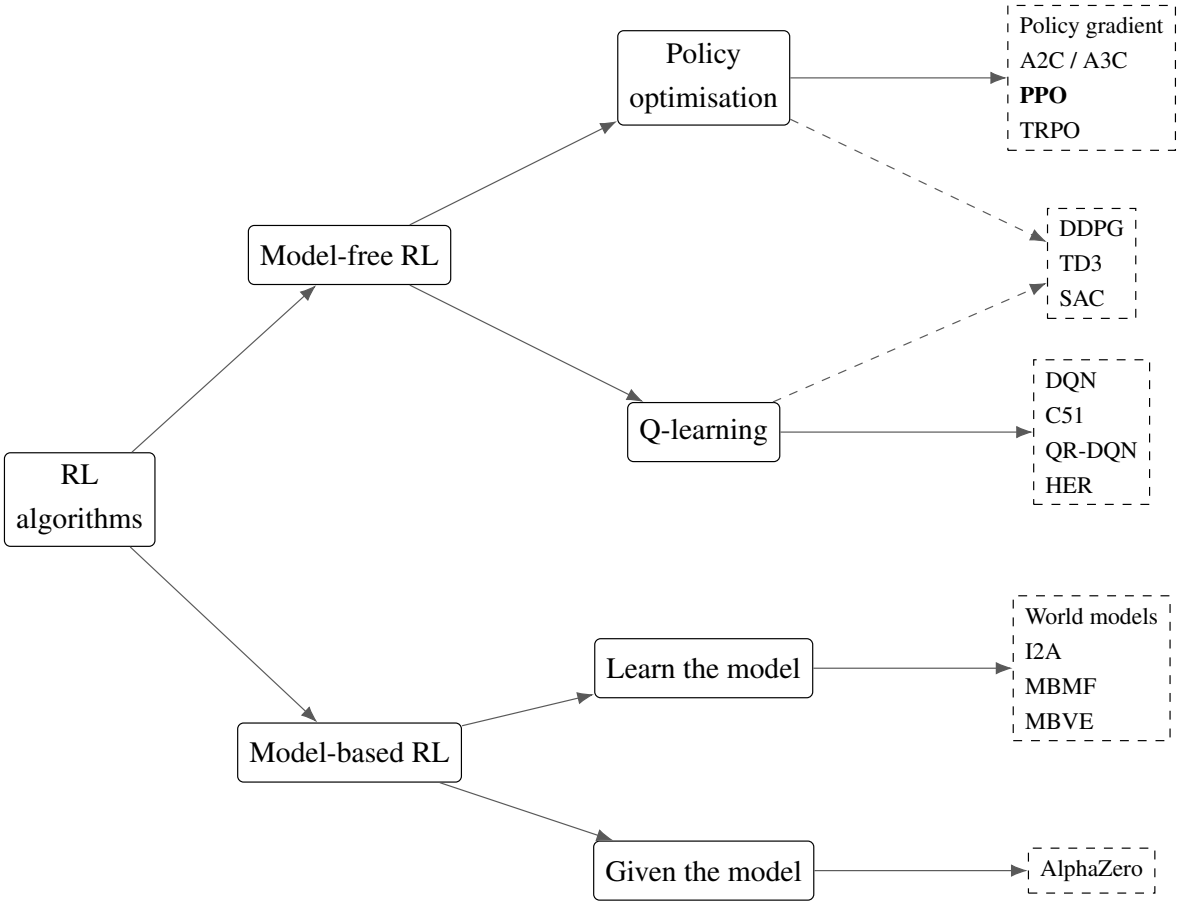

Figure 3.2. A taxonomy of reinforcement-learning algorithms. Redrawn from the taxonomy given in OpenAI's Spinning Up in Deep Reinforcement Learning [22]; the figure is an original rendering of that classification, not a reproduction of the original artwork. The algorithm used in this thesis is shown in bold.

Algorithms also divide by what data they may learn from. An *on-policy* method updates the policy using only transitions collected by the policy as it currently stands, discarding them once an update changes the policy. An *off-policy* method keeps a replay buffer and reuses transitions from earlier versions of the policy. Table 3.4 sets the two side by side.

Table 3.4. On-policy and off-policy methods compared.

Property	On-policy	Off-policy
Data used for an update	Only transitions from the current policy	Any past transition, from a replay buffer
Data reuse	Discarded after one update	Reused many times
Sample efficiency	Lower	Higher
Stability and reproducibility	Higher; optimises the distribution it visits	Lower; distribution shift between behaviour and target policy
Typical bottleneck	Gradient computation	Replay and value-estimation error
Suits	Cheap, massively parallel simulation	Expensive interaction, physical hardware
Example algorithms	**PPO**, TRPO, A2C / A3C, vanilla policy gradient	SAC, TD3, DDPG, DQN
Used in this thesis	Yes, PPO and its Lagrangian variant	No; retained as future work

This study is on-policy, and the reason is that its samples are cheap. At 4096 environments running in parallel on one GPU the bottleneck is the gradient computation, not the collection of experience, so the sample efficiency an off-policy method would buy has little to pay for itself with. Both families are established on robotic grasping [16], so this is a positioned choice rather than the only available one, and Chapter 6 returns to the off-policy comparison as the recognised complement to what is reported here. The particular on-policy algorithm used, and its constrained variant, are developed in Section 3.8, once the problem they solve has been stated.

Key points

- This work is model-free and on-policy, in the policy-optimisation branch of Figure 3.2.
- On-policy methods discard data after one update but stay stable and reproducible; off-policy methods reuse a replay buffer and are far more sample efficient.
- The choice here follows from cheap samples, not from off-policy methods being unsuitable. At 4096 parallel environments the gradient step is the bottleneck, not data collection.
- An off-policy comparison arm was pre-registered and cut on schedule grounds, so it is reported as a limitation rather than quietly omitted.

3.4 Problem formulation

The grasping task is posed as a constrained Markov decision process, or CMDP [11]. A standard Markov decision process is the tuple (S, A, P, r, γ) over a state space S, an action space A, a transition kernel P giving the probability of the next state under a chosen action, a reward function r and a discount factor γ. A CMDP augments that tuple with a cost function c, which scores each transition for hazard exactly as r scores it for progress, and a budget d. It asks for the policy that maximises expected discounted return subject to the expected accumulated cost staying within budget:

$$\max_{\pi} \mathbb{E}\left[\sum_{t} \gamma^{t} r(s_t, a_t)\right] \quad \text{subject to} \quad \mathbb{E}\left[\sum_{t} c(s_t, a_t)\right] \leq d. \tag{3.1}$$

The formulation separates *what the robot should achieve*, which is to lift the cube and bring it to a commanded pose, from *what it must not do*, which is to pass through configurations where the arm loses controllability or approaches its mechanical limits. The first is encoded in r and the second in c, and the two never mix. This is the structural property the whole comparison in Chapter 4 rests on: because no safety term appears in the reward, any safety difference measured between a constrained agent and an unconstrained one is attributable to the constraint machinery and not to a reward the two agents were optimising differently.

Three properties of Equation 3.1 shape how the results must be read, and stating them here avoids over-claiming later.

The constraint binds an **expectation, not a maximum**. A policy satisfying it is one whose *average* accumulated cost over episodes falls within d, which permits individual episodes to exceed the budget provided others compensate. A hardware safety case therefore has to be argued in terms of expected exposure over many operations, not in terms of a guaranteed worst case [14], and Chapter 4 reports a counter-result where the single worst episode is no better under the constraint than without it.

The cost is **undiscounted and episodic**. Where the return in Equation 3.1 is discounted by γ, the constrained quantity is a plain sum over a fixed 250-step episode. Hazard late in an episode therefore counts as heavily as hazard early in it, which is the correct treatment for a physical limit but ties the numerical value of d to the episode length. Changing the episode duration rescales the constraint and voids its calibration, a point Section 3.9 returns to.

The budget is **a single scalar over an aggregated cost**. Section 3.7 sums three separate hazard terms into one number before the constraint sees it, so one multiplier governs all three and the agent is free to trade between them. This keeps the optimisation simple at the price of not being able to state a separate guarantee per hazard, and the three terms are therefore reported separately even though they are constrained jointly.

Solving Equation 3.1 directly is awkward, since the constraint couples every timestep of every episode. The standard treatment, and the one used here, converts it into an unconstrained saddle-point problem through a Lagrange multiplier, which Section 3.8 develops. The remainder of the chapter fills in each element of the tuple in turn: the simulation environment and the MDP itself (Section 3.5), the software that implements them (Section 3.6), the safety cost that instantiates c (Section 3.7), the algorithm that solves the CMDP (Section 3.8), the calibration that makes the constraint meaningful rather than decorative (Section 3.9), and the training and evaluation protocol (Sections 3.10–3.11).

Key points

- The task is a CMDP: maximise expected return subject to expected accumulated cost staying within a budget d.
- Reward and cost are kept in separate channels, which is what makes the Chapter 4 comparison attributable to the constraint.
- The constraint binds an average, not a worst case, so the safety claim is about expected exposure rather than a guarantee.
- The cost is undiscounted over a fixed-length episode, so d is only meaningful at the episode length it was calibrated against.

3.5 The grasping task

The task runs on the platform of Section 3.1, is trained with the `rsl_rl` 3.0.1 library [23], and is registered as `Isaac-Lift-Cube-UR5e-v0`. It is retargeted from Isaac Lab's Franka cube-lift environment onto the UR5e of Section 3.2, equipped with a Robotiq 2F-85 gripper. Only the robot articulation, the action space and the end-effector frame are replaced, so the reward shaping and the task structure of a well-tested base environment are preserved rather than rewritten.

The arm is controlled by joint-position commands, the gripper by a single binary open/close command on the Robotiq actuation joint. The object is a rigid cube (Isaac's DexCube asset scaled to 0.8) initialised on a table in front of the arm, and its target is a goal pose sampled once per episode.

Contact-based grasping does not hold the cube in this asset. A gripper diagnostic traced the cause: in the converted USD file every gripper rigid body is reported at the wrist origin rather than distributed along the gripper body, while the pad-to-pad separation is reproduced correctly. The linkage is therefore intact but its transform relative to the wrist is degenerate, so finger contact cannot resolve where the geometry is drawn, which accounts for the cube falling through the fingers regardless of clamp force.

Because the Layer 1 result, the safe-RL comparison delivered by this thesis, is independent of the grasp mechanism, the gripper is modelled as a lumped mass at the wrist flange and contact grasping is replaced by a *proximity-weld* abstraction. The tool centre point is defined kinematically as a fixed 160 mm offset along the `wrist_3_link` approach axis (the position a correctly mounted 2F-85 pad midpoint would occupy), and when the policy commands a close action while the cube lies within a 6 cm tolerance of that frame, the cube latches and its pose tracks the end-effector each control step (with its velocity zeroed); an open command releases it and physics resumes. This makes "grasp-and-lift" reliable so that both the baseline and the constrained policy can learn the task, and defers realistic finger contact to the Layer 3 sim-to-real work.

Two consequences follow. The lumped-mass gripper alters the wrist inertia relative to a fully articulated model, but the same asset is used for every run, so this affects all arms identically and cannot confound the comparison. And with self-collision disabled and the collision term reduced to a table-plane keep-out, learned configurations are not guaranteed to be self-collision-free on hardware. Both are revisited in the Limitations discussion.

3.5.1 State, action, and reward

Table 3.5 gives the observation, the action and the episode structure.

Table 3.5. Observation, action and episode.

Element	Definition
Observation	Arm joint positions and velocities (relative to defaults), cube position in the base frame, commanded goal position, previous action. All terms clamped to a finite range
Action	6 target arm-joint positions (scale 0.5) + 1 binary gripper command
Episode	5.0 s = 250 control steps at 50 Hz, decimation 2 physics steps per action
Goal sampling	Once per episode, uniform over x [0.22, 0.60], y [−0.30, 0.30], z [0.10, 0.50] m
Cube reset	Randomised within a small planar region

Two of those rows carry a decision rather than a value. The object pose is given to the policy directly instead of being estimated from an image, which isolates the safe-RL contribution from perception; closing that loop with vision was Layer 2 of the original programme, not attempted here. And the goal box is bounded by reachability, not convenience, as Figure 3.3 shows. Clamping the observations is a numerical safeguard that stops one transiently unstable environment from corrupting the whole on-policy batch.

The reward is the shaped sum inherited from the base lift task, with weights retuned for this

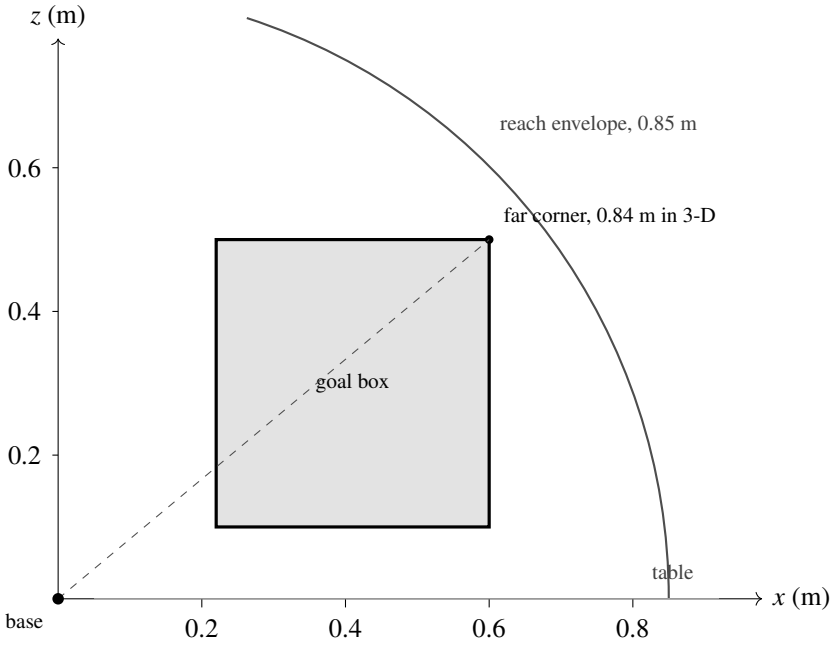

Figure 3.3. The goal-sampling box against the arm's reach envelope, seen in the vertical plane through the base. The far corner of the three-dimensional box, which includes the $y = \pm 0.30$ m extent not visible in this section, lies 0.84 m from the base against a rated reach of 0.85 m.

goal box (Table 3.6). All three lift-dependent terms gate on the cube having climbed half of the distance from the table to the goal height sampled for that episode, rather than on a fixed absolute height, because a fixed threshold scales badly once the box spans 0.10 m to 0.50 m in z. Shaped distance-and-lift rewards of this form, tuned per task rather than derived, are the standard construction in learned contact-rich manipulation [15], so the design is conventional and only its safety treatment is not.

Table 3.6. Reward terms.

Term	Form	Weight
Reaching	tanh kernel on end-effector-to-object distance, $\sigma = 0.1$	1
Lifting	Gated on 50 % of the climb to the goal height	10
Goal tracking, coarse	tanh kernel on cube-to-goal distance, $\sigma = 0.3$	15
Goal tracking, fine	tanh kernel on cube-to-goal distance, $\sigma = 0.05$	5
Action rate	Quadratic penalty	-10^{-4}
Joint velocity	Quadratic penalty	-10^{-4}
Safety	**None.** Safety is a separate cost channel	**n/a**

The last row is the one that matters for the comparison. The reward contains no safety term

at all, so safety is expressed entirely through the cost channel of Section 3.7 and the Results chapter can attribute any safety difference to the constraint. Table 3.7 collects the scene settings not already given above.

Table 3.7. Scene and simulation settings.

Item	Setting
Simulator	Isaac Sim 5.0.0, PhysX, GPU-resident
Framework	Isaac Lab 2.3.0, manager-based
Task	`Isaac-Lift-Cube-UR5e-v0`, retargeted from the Franka cube-lift task
Robot	UR5e, 6 revolute joints, fixed base at the environment origin
End effector	Robotiq 2F-85, modelled as a lumped mass at the wrist flange
Grasp model	Proximity weld, 160 mm TCP offset, 6 cm latch tolerance
Object	DexCube asset, scale 0.8, reset on the table in front of the arm
Self-collision	Disabled; collision cost is a table-plane keep-out
Parallel environments	4096 (training), 128 (evaluation)

3.6 Software framework

Every piece of code written for this thesis lives in one Python package, `ur5_grasp`, installed beside Isaac Lab rather than inside it. Keeping it outside the simulator's own source tree has two practical consequences. The environment, the safety cost and the constrained algorithm are versioned and frozen as a single unit, which is what makes a run reproducible. It also keeps the boundary visible between what this work contributes and what it inherits from the framework. Table 3.8 lists the modules.

The safety cost is computed by `SafetyCostComputer`. It resolves the arm joint indices, the monitored body indices and the Jacobian body-axis index once on the first call and caches them, and all arithmetic is batched across environments with nothing copied back to the host. At 4096 environments a per-step host synchronisation would cost more than the physics step it is measuring. The class returns the aggregate cost that the constraint acts on, and alongside it a mean value and a binary violation indicator for each of the three terms, so the constraints can be reported separately even though a single multiplier governs them jointly.

The four `safe_rl` modules extend `rsl_rl` [23] rather than replacing it with a different library, and two details of how they attach matter later. `ActorCriticCost` builds the cost critic *after* the base class has finished, so the actor and reward critic draw their initial weights

Table 3.8. The ur5_grasp package.

Module	Role
robots/ur5e_robotiq.py	UR5e articulation, actuator gains, joint speed and effort limits
tasks/lift/ur5e_lift_env_cfg.py	Task configuration: action space, goal-sampling box, reward terms
tasks/lift/ur5e_lift_env.py	Environment class, safety thresholds, per-step cost and safety/* logging
tasks/lift/rewards.py	Goal-relative lift gate used by the three lift-dependent reward terms
safe_rl/costs.py	SafetyCostComputer: the three cost terms of Section 3.7
safe_rl/actor_critic_cost.py	Actor, reward critic and cost critic
safe_rl/rollout_storage_cost.py	Rollout buffer carrying the cost stream and cost advantages
safe_rl/ppo_lagrangian.py	Combined advantage, dual update, partitioned gradient clip
safe_rl/lagrangian_runner.py	Runner that assembles the constrained agent
tasks/lift/agents/	Per-arm hyperparameter configurations (baseline, control, cPPO)
scripts/train.py	Training entry point for every arm
scripts/eval_policy.py	Evaluation of a frozen checkpoint (Section 3.11)
tools/	Asset builders, gripper diagnostics, run summarisers, regression tests

from the same random state as the unconstrained baseline's. LagrangianRunner replaces exactly one method of the stock runner, _construct_algorithm, which is necessary only because the stock runner resolves class names inside its own module namespace, where classes defined outside the library are invisible. Everything else, the rollout loop, the logging and the checkpointing, is inherited unchanged.

Both entry points are single scripts, so the arms cannot diverge by accident: the baseline and the constrained agents are launched by the same command with one configuration flag changed. Each run also writes its resolved environment and agent configuration to disk beside its checkpoints, so two runs are compared by differencing their dumped configurations rather than by trusting that the same flags were passed.

One implementation detail of that extension has to be stated here because the experimental design depends on it. Stock PPO applies a single gradient-norm clip across every parameter handed to the optimiser. In a constrained agent that parameter list also holds the cost critic, so the cost critic's gradients enter the norm and shrink the actor's step even when the multiplier is zero, which makes the constrained agent an optimiser change rather than purely

a constraint. The clip is therefore partitioned, the actor and reward critic in one group and the cost critic in another, so the baseline half of the network is treated exactly as the unconstrained agent's is. Because a correction of this kind cannot be verified by reading the code, a control arm was added that carries the cost critic with its multiplier ceiling pinned to zero. Section 4.2 reports that verification and the withdrawn result that prompted it.

Key points

- All contributed code sits in one package outside the simulator tree, so the environment, the cost and the algorithm are frozen together.
- The constrained agent is a thin extension of the same trainer the baseline uses, which is what lets a measured difference be attributed to the constraint rather than to two different implementations of PPO.
- The gradient clip is partitioned so that the cost critic cannot alter the actor's step, and a control arm exists to verify that it does not.

3.7 The safety cost function

Safety is encoded by a per-step cost, computed by the cost class of Section 3.6 from three geometric-kinematic terms. Each term is zero while the arm is safe and grows smoothly as the arm enters a danger zone; smoothness matters because it gives the cost critic (Section 3.8) a usable gradient. The three terms are summed into a single aggregate cost so that a single Lagrange multiplier governs the whole constraint, and each term additionally exposes a mean value and a binary violation indicator for reporting.

Table 3.9 defines the three terms.

Table 3.9. The three safety cost terms. Each is summed over the bodies or joints it monitors, and the three are added with unit weights.

Term	Per-step penalty	Threshold	Status
Collision keep-out	max(z_floor − z, 0), summed over forearm, wrist-1 and wrist-3	z_floor = 0.05 m	Monitored, inactive
Joint-limit margin	clamp(1 − clearance / margin, 0, 1), summed over the 6 arm joints	margin = 0.175 rad (10.0°)	**Active**
Singularity floor	clamp(1 − w / w_floor, 0, 1) on the manipulability measure w	w_floor = 0.06	**Active**

The third term is the one this study is designed around. Manipulability is where the thesis makes contact with the literature, since the nearest prior work places the same quantity in the reward rather than in a constraint, and the research question is posed about it. It is not, however, the only active term, and it is not uniformly the dominant one: Section 3.9 shows that

which term consumes the budget varies from seed to seed, so calling either one *the* operative constraint would misdescribe at least one of the reported runs. The Yoshikawa manipulability measure w = $\sqrt{\det(JJ^\top)}$ [17] is computed from the 6×6 end-effector Jacobian J of the arm. A small w means a near-singular configuration, one in which the arm momentarily loses the ability to move in some Cartesian direction and a modest Cartesian command produces large, jerky joint velocities. The Jacobian body-axis index is resolved automatically to accommodate the fixed-base articulation, for which PhysX omits the root body from the Jacobian, and correctness was confirmed at calibration by a smooth, small-positive distribution of w.

Figure 3.4 shows the shape of that penalty against the baseline's own distribution of w. The ramp is what makes the term learnable: a binary violation flag would be zero almost everywhere and would give the cost critic no gradient to descend, whereas this form tells the agent how close it is to trouble as well as whether it is in it.

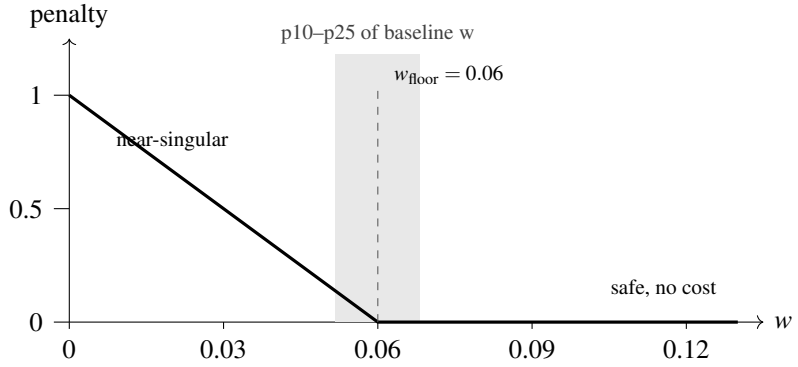

Figure 3.4. The singularity penalty against the manipulability measure w. The floor sits at 0.06, inside the tenth-to-twenty-fifth percentile band of the unconstrained baseline's own distribution, so the baseline pays a cost on roughly a fifth of its steps.

Two details of the other terms are worth stating. The collision floor is a 5 cm standoff above the table rather than the table surface itself, so the term begins to cost before a link penetrates anything; with self-collisions disabled and no contact sensors present, this geometric proxy is the honest and inexpensive choice. The joint-limit penalty has the same ramp shape, zero until a joint enters the final margin band and rising linearly to one at the limit.

The aggregate per-step cost is c = c_collision + c_joint + c_singularity, and its undiscounted sum over an episode is the quantity constrained against the budget d.

Key points

- Three hazards are scored: proximity to the table, proximity to a joint limit, and proximity to a kinematic singularity.
- Every term is smooth rather than a switch, because the cost critic needs a gradient to learn from. A binary violation flag would give it none.

- The singularity term uses the Yoshikawa manipulability measure and is the term the study is designed around, but the joint-limit term is active too, and which of the two carries the budget varies by seed. Section 3.9 gives the measured split.
- The three are summed into one number before the constraint sees it, so one multiplier governs all three, but each is logged separately for reporting.

3.8 Policy optimisation: PPO and cPPO

This section develops the algorithm that solves the CMDP of Section 3.4. It is built in two stages, because the constrained method changes exactly one term of the unconstrained one and the change is far easier to follow once the unmodified objective is on the page. Proximal Policy Optimisation, PPO, is the unconstrained baseline [12]; the constrained arm, written cPPO throughout, is its Lagrangian variant.

A policy-gradient method improves a parameterised policy π_θ by ascending an estimate of the gradient of expected return, in which each action is weighted by its advantage $\hat{A}_t$, that is, by how much better it was than the policy's average behaviour in that state. Ascending that estimate directly is unreliable, because it is only valid near the policy that collected the data: a step large enough to be useful is often large enough to move the policy somewhere the estimate no longer describes. Trust-region methods bound the policy change explicitly, at the cost of a second-order optimisation. PPO reaches the same end with a first-order method by building the bound into the objective.

3.8.1 The clipped surrogate objective

Let $\pi_{\theta_{\text{old}}}$ be the policy that collected the current batch. PPO works with the probability ratio between the candidate policy and that one,

$$r_t(\theta) \;=\; \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}, \qquad r_t(\theta_{\text{old}}) = 1. \tag{3.2}$$

The ratio measures how much more, or less, likely the candidate policy is to have taken the action that was actually taken. An unconstrained surrogate would maximise $r_t(\theta)\hat{A}_t$, which grows without bound as the policy moves further from the one that generated the data. PPO instead maximises

$$L^{\text{CLIP}}(\theta) \;=\; \hat{\mathbb{E}}_t\Big[\min\Big(r_t(\theta)\hat{A}_t,\; \text{clip}\big(r_t(\theta),\, 1-\varepsilon,\, 1+\varepsilon\big)\hat{A}_t\Big)\Big], \tag{3.3}$$

with clipping range $\varepsilon = 0.2$ in this work. The behaviour of Equation 3.3 is asymmetric in a way worth stating plainly. Where the advantage is positive, the action was better than average

and the objective would reward pushing r_t up, but the clip caps the gain at $1+\varepsilon$, so there is no incentive to move the policy further than that. Where the advantage is negative, the same cap applies at $1-\varepsilon$ in the other direction. The minimum of the clipped and unclipped terms makes the objective a pessimistic bound on the unclipped one, so the update never benefits from a large policy change, only from a well-directed small one. A single scalar therefore does the work a trust region does, without a constrained optimisation.

3.8.2 Advantage estimation and the full objective

The advantage itself is estimated by generalised advantage estimation over a truncated rollout of length T. Writing the temporal-difference residual as

$$\delta_t \;=\; r_t + \gamma V(s_{t+1}) - V(s_t), \tag{3.4}$$

the estimate is the exponentially weighted sum

$$\hat{A}_t \;=\; \sum_{l=0}^{T-t-1} (\gamma\lambda_{\mathrm{GAE}})^l \, \delta_{t+l}, \tag{3.5}$$

in which γ discounts future reward and λ_{GAE} sets where the estimate sits between a one-step bootstrap, at $\lambda_{\mathrm{GAE}} = 0$, which has low variance and high bias, and a full Monte-Carlo return, at $\lambda_{\mathrm{GAE}} = 1$, which has the opposite. This work uses $\gamma = 0.98$ and $\lambda_{\mathrm{GAE}} = 0.95$. The subscript is carried deliberately: from Section 3.8.3 onward an unsubscripted λ always means the Lagrange multiplier, which is a different quantity entirely.

Because the value function V appearing in Equation 3.4 is itself learned, and because a policy that collapses to a single action stops exploring, the quantity actually optimised combines three terms:

$$L(\theta) \;=\; \hat{\mathbb{E}}_t\Big[L_t^{\mathrm{CLIP}}(\theta) \;-\; c_1 \left(V_\theta(s_t) - V_t^{\mathrm{targ}}\right)^2 \;+\; c_2 S\big[\pi_\theta\big](s_t) \Big], \tag{3.6}$$

where the second term fits the value network to its bootstrapped target with coefficient $c_1 = 1.0$, and the third is an entropy bonus with coefficient $c_2 = 0.006$ that keeps the policy stochastic for long enough to explore. Each batch of experience is then used for several epochs of minibatch gradient steps, five epochs of four minibatches here, which recovers part of the sample efficiency an on-policy method gives up by discarding its data. The implementation used in this work additionally adapts the learning rate between iterations to hold the observed Kullback-Leibler divergence between successive policies near a target of 0.01, which is a second, softer guard on the same failure the clip addresses.

3.8.3 *The constrained variant*

cPPO changes Equation 3.6 in one place and leaves the rest alone. Figure 3.5 shows the whole agent, with everything cPPO adds over the baseline drawn inside the dashed region: a second critic, a scalar Lagrange multiplier, and the dual-ascent loop that sets the multiplier from measured violation rather than from a designer's guess. The method is PPO-Lagrangian as specified and benchmarked by Ray et al. [8], within the constrained policy optimisation family founded by Achiam et al. [5]. Their benchmark is also the reason the Lagrangian variant is preferred here over CPO itself, which it reports as the weaker of the two on the same tasks.

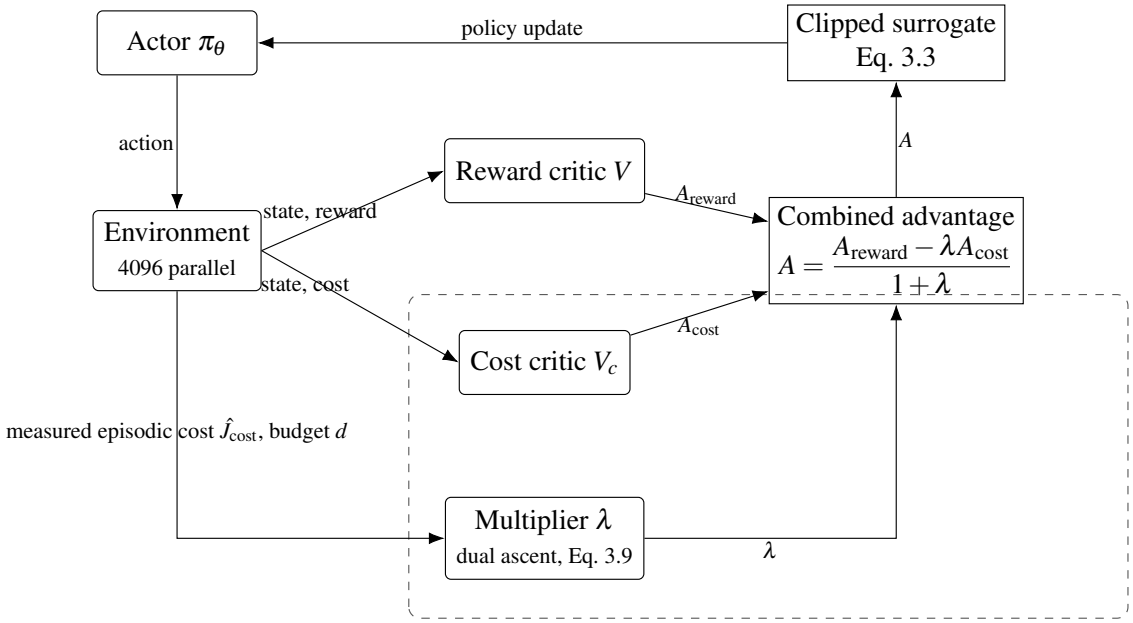

Figure 3.5. The cPPO agent. The dashed region marks what the constrained arm adds to the baseline: a cost critic and a multiplier updated by dual ascent on measured violation. Their only point of entry into the policy update is the combined advantage, and pinning λ to zero recovers the baseline exactly.

The method converts the constrained problem into the saddle-point objective

$$\max_{\pi} \min_{\lambda \geq 0} \quad J_{\text{reward}}(\pi) - \lambda \left(J_{\text{cost}}(\pi) - d \right), \tag{3.7}$$

where λ is a non-negative Lagrange multiplier acting as an automatically-tuned penalty on constraint violation. The implementation, whose modules are listed in Table 3.8, follows the textbook separate-critic variant: in addition to the usual value network that estimates future reward, a second critic estimates future cost, so reward and cost advantages are learned independently. Cost advantages are computed by generalised advantage estimation with its own discount and smoothing (γ_cost = 0.98, λ_GAE = 0.95).

At each policy-update step the reward and cost advantages are combined into a single La-

grangian advantage,

$$A = \frac{A_{\text{reward}} - \lambda A_{\text{cost}}}{1 + \lambda}, \tag{3.8}$$

which is then used in the standard clipped PPO surrogate; the division by (1 + λ) keeps the effective step size stable as λ grows. The multiplier itself is updated once per iteration by projected dual ascent on the measured mean episodic cost of the just-collected rollout,

$$\lambda \leftarrow \text{clip}\big(\lambda + \eta\,(\hat{J}_{\text{cost}} - d),\ 0,\ \lambda_{\max}\big), \tag{3.9}$$

with dual step η = 0.035, initial λ = 0, and ceiling λ_max = 100. Intuitively, whenever the policy exceeds its cost budget the multiplier rises and unsafe actions are penalised more heavily; once the policy is comfortably under budget the multiplier decays back toward zero. The multiplier tunes itself during training and is never hand-set.

Setting $\lambda = 0$ in Equation 3.8 returns $A = A_{\text{reward}}$, so the unconstrained baseline is this same algorithm with the multiplier pinned at zero. Building the constrained agent as a thin extension of the baseline, rather than adopting a separate safe-RL library, is a deliberate methodological choice: it guarantees that the two share an identical trainer, network architecture and hyperparameters, so any measured difference is attributable to the safety constraint alone and not to two dissimilar PPO implementations.

Both agents use the same actor and critic multilayer perceptrons with hidden layers of 256, 128 and 64 units and ELU activations, and the same optimisation hyperparameters, summarised in Table 3.10.

Table 3.10. Shared training hyperparameters.

Hyperparameter	Value
Parallel environments	4096
Rollout length (steps/env)	24
Iterations	1500
Actor / critic hidden layers	[256, 128, 64], ELU
Clip parameter	0.2
Entropy coefficient	0.006
Learning epochs / mini-batches	5 / 4
Learning rate (adaptive, target KL 0.01)	1×10^{-4}
Discount γ / λ_{GAE}	0.98 / 0.95
Max gradient norm (per group)	1.0
cPPO cost budget d (`cost_limit`)	25, and 15 for the tighter arm
cPPO dual step η / λ ceiling	0.035 / 100
cPPO cost discount / λ_{GAE}	0.98 / 0.95

Key points

- PPO's central idea is a probability ratio between the new policy and the one that collected the data, clipped so that a large policy change can never be rewarded.
- The clipped objective is a pessimistic bound on the unclipped one, which is how a first-order method achieves what a trust region does second-order.
- Advantages come from generalised advantage estimation, where λ_{GAE} trades bias against variance. It is not the Lagrange multiplier, which shares the letter.
- cPPO changes exactly one quantity in that objective, the advantage. The clip, the value loss, the entropy bonus and the epoch structure are identical to the baseline, and the baseline is this algorithm with the multiplier pinned at zero.
- The multiplier is set by dual ascent on measured violation, so the safety-versus-task weight is never hand-tuned. This is the property a shaped reward cannot offer.

3.9 Constraint calibration and cost budget

A safety benchmark is only meaningful if the unconstrained baseline actually violates the constraint; otherwise the constrained policy has nothing to demonstrate. All three thresholds and the budget were therefore measured from a converged 1500-iteration unconstrained agent, run against the final environment immediately before the code was frozen. Calibrating against a converged policy rather than a short probe matters here, because a policy fifty iterations into training has barely begun to reach and its cost distribution says little about the one that would be deployed. Table 3.11 gives what was measured and what each measurement settled.

Table 3.11. The calibration probe: what was measured on one converged unconstrained run, and what each measurement settled. The probe is a single run and its cost composition is not representative of the five reported seeds, for which see Table 3.12.

Quantity	Measured on the probe	Outcome
Manipulability w	min 0.0171, mean 0.0750, max 0.1109; p10 = 0.0517, p25 = 0.0682	Floor set to 0.06, about p18, so the baseline violates on roughly a fifth of steps
Joint-limit clearance	33.7 % of steps inside the margin; minimum clearance reaches the soft limit	Margin 0.175 rad is **active**, not monitored-but-satisfied as in earlier drafts
Monitored-link height	Minimum 0.0896 m, about 44 mm above the floor	Collision floor 0.05 m stays inactive
Cost composition	Joint-limit $\approx$ 86 %, singularity $\approx$ 14 %, collision 0 %	Two constraints share one budget
Natural episodic cost	$\approx$ 105	Budget d = 25, a $\approx$ 76 % reduction against this run

The probe settled the thresholds, and that is all it settled. It is one run, and its cost composition turns out not to describe the five baseline seeds that are actually reported. Table 3.12 gives those, measured from the same training logs Chapter 4 draws on.

Table 3.12. How the unconstrained baseline's episodic cost divides between the two active terms, per seed, as a tail mean over the final tenth of training. The collision term is zero on every seed. Chapter 4 reports the episodic cost totals themselves.

Seed	Singularity	Joint-limit
1	15 %	**85 %**
3	**100 %**	0 %
4	**100 %**	0 %
52	**100 %**	0 %
54	49 %	51 %
Mean	78 %	22 %

Three things follow, and the third is the one that matters. First, the manipulability floor was raised to 0.06 from an earlier value of 0.045, which had been set against a narrower goal-sampling box and no longer produced a violation rate inside the intended band once the workspace was widened. Second, joint-limit proximity is genuinely active rather than the monitored-but-satisfied term it was in earlier drafts, so two constraints share one budget and one multiplier and the agent is free to trade between them. Third, and least expected, **the split between them is not a property of the task but of the seed**. Three of the five baseline runs spend their entire budget on near-singular configurations and never approach a joint limit; one spends most of it on joint-limit proximity instead. Averaging gives 78 per cent against 22, but the average describes none of the individual runs well. That instability is a result in its own right and Chapter 4 develops it; here it is a warning against reading either term as *the* constraint the method acts on.

The budget itself is unaffected by any of this. Holding d = 25 against a mean natural cost of about 82 asks for roughly a 70 per cent reduction, and against the probe's 105 for about 76 per cent. It is an undiscounted sum over a per-step cost across a fixed 250-step episode, so it is tied to the five-second episode length: changing that length would rescale the constraint and void the calibration.

3.10 Training protocol

Table 3.13 states the protocol as run. The one thing it cannot state is what a successful run looks like while it is in progress. The healthy Lagrangian signature to be verified during training is that the mean episodic cost trends down toward the budget, the multiplier rises while over budget and then settles without pinning at its ceiling, the reward continues to

improve, and the singularity-violation rate falls below the baseline.

Table 3.13. Training protocol.

Item	Setting
Arms	3: PPO (baseline), cPPO at d = 25, cPPO at d = 15
Seeds per arm	5: 1, 3, 4, 52, 54
Trained policies	15
Iterations	1500 per run
Parallel environments	4096
Rollout length	24 steps per environment per iteration
Throughput	$\approx 2 \times 10^5$ environment steps/s on an RTX 5090
Library	`rsl_rl` 3.0.1, PyTorch 2.7, CUDA 12.8
Execution	Headless, monitored through TensorBoard
Launch	One script, arm selected by agent-config entry point
Logging	Separate experiment directory per arm, so curves overlay directly
Verification	All 15 final checkpoints confirmed present on disk, not inferred from clean logs

3.10.1 Experimental arms and seeds

Three arms were trained, listed in Table 3.14. They differ only in the two fields shown; network architecture, hyperparameters, environment and trainer are identical across all three. The second constrained budget is a sensitivity check on the calibration rather than a separate method: 15 binds more deeply than 25 on the seeds where either binds at all, so the pair shows whether the effect of the constraint scales with how hard it is applied.

Table 3.14. The three experimental arms.

Arm	`cost_limit`	$\lambda_{\max}$	Reported in this thesis as
`ctrl`	25	0	PPO (baseline)[1]
`cppo`	25	100	cPPO, d = 25
`cppo15`	15	100	cPPO, d = 15

Each arm was trained on five random seeds, 1, 3, 4, 52 and 54, giving fifteen trained policies. Reporting five seeds and their dispersion, rather than a single run or a mean alone, follows the reproducibility argument made by Henderson et al. [6], who show that deep policy-gradient

[1]The control arm carries the cost critic with its multiplier ceiling pinned to zero, so its policy update is stock PPO. A plain PPO arm was trained on the same seeds and its stored network weights proved byte-identical to the control arm's, so the control arm stands in for it and the plain arm is not reported separately. The verification is given in Section 4.2.

results on continuous control can differ qualitatively between seeds of the same algorithm and that a mean over too few runs can describe a policy none of them produced. Dispersion across seeds is therefore reported as a result in its own right in Chapter 4, not as an error bar attached to one.

3.11 Evaluation protocol

Every reported number is measured after training on the frozen policy, not read from training-time telemetry. This distinction is a correction rather than a preference. An earlier version of this work reported safety percentages taken from the TensorBoard scalars of a policy that was still exploring and still learning, which describes neither the policy that would be deployed nor any policy at all. Each checkpoint is therefore replayed by `eval_policy.py` with the actor set to its deterministic mean action, exploration noise off and observation corruption disabled.

Table 3.15 states the protocol. Three of its choices carry an argument that the rows cannot.

Evaluation seeds are disjoint from training seeds so that variation caused by the exam is separable from variation caused by which policy was learned, and the two turn out to differ by more than an order of magnitude. Goal-reach is reported with the full distribution of final goal distances rather than on its own, because the weld writes the cube's pose to the tool frame at every step, so the 1 cm test reduces to whether the tool centre point reached the goal: a converged policy passes it on almost every episode and an unconverged one fails on almost every episode, which makes a single threshold saturate and hide exactly the differences a distribution shows. And safety is counted per episode rather than averaged per step, so one dangerous excursion stays visible instead of being diluted across 250 steps. The resulting comparison is presented in Chapter 4.

Table 3.15. Evaluation protocol.

Item	Setting
Policy	Frozen checkpoint, deterministic mean action, exploration noise off
Observation corruption	Disabled
Evaluation seeds	101, 102, 103, disjoint from the training seeds
Episodes	1000 per checkpoint per evaluation seed
Parallel environments	128
Episodes per policy	3000
Episodes per arm	15,000
Lift success	Cube past 50 % of the climb to that episode's goal height
Goal-reach success	Lifted cube within 1 cm of the commanded goal pose
Reported alongside	Full distribution of final goal distances, since the weld makes a single threshold saturate
Safety, per episode	Episodic cost; singularity crossings ($w < 10^{-4}$); episodes touching the joint-limit margin
Constrained arms only	Terminal multiplier, episodic-cost trajectory

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Experimental design and provenance

The results in this chapter come from a single frozen state of the code base. Freezing the environment, the cost function and the calibrated thresholds before the first training run of the comparison, rather than after, is a deliberate part of the fairness protocol: it makes it impossible for a threshold to be adjusted in response to a result that has already been seen. All runs use the frozen weld environment `Isaac-Lift-Cube-UR5e-v0` described in Chapter 3.

Three arms are reported, as set out in Section 3.10.1. The `ctrl` arm carries the cost critic with its multiplier ceiling pinned to zero, so its policy update is stock PPO; it is labelled PPO (baseline) throughout.[1] The `cppo` arm is the constrained PPO-Lagrangian agent [8, 5] at an episodic cost budget of 25, the value inherited from the Safety Gym reference configuration. The `cppo15` arm is the same agent at a budget of 15, which is well below the baseline's natural operating cost on every seed and is therefore actively binding rather than borderline.

Each arm was trained on five random seeds (1, 3, 4, 52 and 54) for 1500 iterations at 4096 parallel environments using `rsl_rl` 3.0.1, giving fifteen trained policies. All fifteen final checkpoints were verified present on disk rather than inferred from the absence of errors in the training logs. Evaluation was then carried out on the frozen, deterministic policy with observation corruption disabled, over three evaluation seeds (101, 102 and 103, disjoint from the training seeds) at 1000 episodes each. Every individual policy is therefore scored over 3000 episodes and every arm over 15,000, for 45,000 evaluation episodes in total. Reporting evaluation statistics from the frozen policy rather than from training-time telemetry is itself a correction carried forward from an earlier iteration of this work, in which safety percentages had been read from the TensorBoard scalars of a still-exploring policy and therefore could not describe the policy that would actually be deployed.

The safety thresholds were calibrated against a converged unconstrained baseline immediately before the freeze, as Section 3.9 records. Two points from that calibration matter for reading what follows. The manipulability floor was raised from 0.045 to 0.06 to restore the baseline violation rate to the intended band. The joint-limit margin was held at 0.175 rad

[1] A plain `ppo` arm was trained on the same five seeds. Its stored weights proved byte-identical to `ctrl`'s, so `ctrl` stands in for it and the plain arm is not reported separately. The verification is Section 4.2; the raw comparison is `Comparison_test/ppo_redundant/README.md`.

but reclassified as active, because the baseline policy spends 33.7 % of its steps inside it, and it is the larger of the two active terms by realised cost. The collision floor of 0.05 m was confirmed inactive and remains so in every result below.

Table 4.1. Experimental configuration.

Item	Value
Task	`Isaac-Lift-Cube-UR5e-v0` (frozen weld environment)
Arms reported	`ctrl` (PPO baseline), `cppo` ($d = 25$), `cppo15` ($d = 15$)
Training seeds	1, 3, 4, 52, 54 (n = 5 per arm; 15 trained policies)
Training budget	1500 iterations, 4096 environments, `rsl_rl` 3.0.1
Evaluation seeds	101, 102, 103 (n = 3, disjoint from training)
Evaluation protocol	1000 episodes per checkpoint × evaluation seed, deterministic policy, observation corruption off
Evaluation episodes	3000 per policy, 15,000 per arm, 45,000 in total
`MANIP_FLOOR`	0.06 (recalibrated from 0.045)
`JOINT_LIMIT_MARGIN`	0.175 rad (active; 33.7 % baseline within-margin rate)
`COLLISION_Z_FLOOR`	0.05 m (confirmed inactive)
`cost_limit`	25 (`cppo`), 15 (`cppo15`)

4.2 Validity check: the control arm reproduces the baseline exactly

An earlier version of this comparison reported a large advantage for the constrained agent that was subsequently traced, in a line-by-line audit against the upstream library, to an implementation artifact rather than to the safety constraint. A single global gradient-norm clip had been applied across all parameters handed to the optimiser. Because the constrained agent carries a third network, the cost critic's gradients entered that norm and scaled down the actor's step on every update. The constrained arm was therefore not PPO plus a constraint, it was PPO with a systematically quieter optimiser, and a quieter optimiser is a well-known stabiliser on shaped-reward manipulation tasks. Because the multiplier had also sat at zero for essentially the whole of those runs, the constraint could not have been responsible for the gap at all. That result was withdrawn in full, the clip was partitioned so that the actor and reward critic receive treatment identical to the baseline, and the `ctrl` arm was added specifically so that the correction could be verified rather than assumed.

The verification is the strongest that this class of check admits, and it now rests on three independent comparisons.

First, the training scalars agree exactly. Across all five seeds, the tail-mean reward, the soft-margin singularity fraction and the minimum-manipulability trace all agree between `ppo_sN` and `ctrl_sN` to every decimal place the logs carry. The agreement extends to chaotic

near-zero quantities where any divergence in trajectory would be expected to show first: the minimum-manipulability tail mean reads 7.419×10^{-6} for both `ppo_s3` and `ctrl_s3`. Agreement at that precision is not what statistical equivalence looks like, and it was therefore treated as a suspected logging fault rather than as a result until it had been checked at the file level.

Second, the possibility that the two arms were reading the same log was excluded directly. The two runs wrote separate files, of different sizes, from separate processes, and the control arm's file is the larger by an amount consistent with its additional cost-critic logging. The two final checkpoints were then opened and every stored tensor compared by content. All 68 tensors constituting `ppo_s1`'s trained actor and reward critic were found byte-for-byte inside `ctrl_s1`'s checkpoint, which carries 100 tensors in total, the remainder being the control arm's own cost critic. Parameter names recovered from the serialised stream confirm that these are the weight and bias tensors of the four actor and four critic layers. Statistical indistinguishability would have been enough for the comparison. What the two arms in fact reached, at the level of the stored weights, is the same policy.

Third, the result reproduces at evaluation time on data the training runs never saw. Every one of the 15,000 evaluation episodes was compared episode by episode between the two arms on final goal distance, episode-minimum manipulability and episodic cost. All 15,000 agree to the precision the per-episode files carry, on all five seeds and all three evaluation seeds. Because the two arms are exactly equal wherever they are compared, they are reported as a single PPO (baseline) column throughout the rest of this chapter.

Interpretation requires one caveat that should be stated rather than glossed. With the clip partitioned and the multiplier pinned to zero, the control arm's actor loss is algebraically identical to the baseline's at every step, so identical *updates* are expected. Identical *weights* additionally require that the random-number stream driving action sampling and minibatch permutation was not displaced by the cost critic's extra parameter draws at construction. The audit had assumed the opposite, and treated the resulting offset as unavoidable seed noise to be absorbed by the control arm. The empirical outcome contradicts that assumption. The effect is verified beyond reasonable doubt, but the mechanism, plausibly separate generator streams for host and device, was not traced in the library source, and is recorded here as an open question rather than asserted as an explanation. Nothing in the chapter depends on the explanation; the observation alone is what licenses the comparison that follows.

4.3 How this comparison must be read

The audit that withdrew the earlier result also fixed the form in which the corrected result may be reported. A direct constrained-versus-baseline number is not permitted as a headline, because such a number silently sums two effects of different kinds. What is reported instead

is the decomposition

$$(\texttt{cppo} - \texttt{ppo}) \;=\; (\texttt{ctrl} - \texttt{ppo}) \;+\; (\texttt{cppo} - \texttt{ctrl}) \tag{4.1}$$

in which the first term is the cost of merely attaching a cost critic, the implementation artifact, and the second is the effect of the constraint itself with everything else held fixed. Only the second term is a safe reinforcement learning result. The first term is the quantity that was mistakenly reported as the second in the withdrawn version of this work.

Section 4.2 establishes that the first term is exactly zero, on every seed and at every point of comparison. That is what makes the decomposition useful rather than merely cautious. Because the artifact term vanishes identically, the whole of any observed difference between a constrained agent and the baseline is attributable to the constraint, and the columns of every table below can be read directly. Had the first term been non-zero, no constrained-versus-baseline number could have been reported at all, because the arms would still have differed by something unnamed.

One further point governs the reading. The two constrained arms differ only in their budget, so the difference between them isolates the effect of *how hard* the constraint is applied, with the algorithm, the environment, the seeds and the network all held fixed. Section 4.8 treats that comparison on its own. The `sac` arm registered in the original plan was not trained, so no off-policy comparison is offered here.

4.4 Task performance

Table 4.2 reports training-time performance as a tail mean over the final tenth of training, with the spread across the five seeds.

Table 4.2. Training-time performance (tail mean over the final 10 % of iterations, mean ± sd across n = 5 seeds).

Quantity	Mean ± sd across n = 5 seeds		
	PPO (baseline)	cPPO, $d = 25$	cPPO, $d = 15$
Mean reward	133.06 ± 1.27	132.18 ± 1.99	**134.09 ± 0.80**
Mean episodic cost	81.86 ± 62.56	14.22 ± 4.38	**10.94 ± 4.30**
`viol_singularity` (step fraction)	0.455 ± 0.338	**0.169 ± 0.043**	0.233 ± 0.088
`viol_joint_limit` (step fraction)	0.076 ± 0.156	**0.000**	**0.000**

On reward the three arms are indistinguishable. The reward decomposition for the $d = 25$ arm is $(\texttt{cppo} - \texttt{ppo}) = 0.000 + (-0.880)$, so the entire difference of about 0.7 % arises from

the constraint and none from the artifact, and 0.880 is smaller than the seed-to-seed standard deviation of either arm. The tighter arm ends *above* the baseline at 134.09, and with the smallest spread of the three. Neither difference should be read as a measured task effect in either direction. What the row supports is that the constraint does not cost reward, and the honest form of that statement is an upper bound rather than a point estimate. Figure 4.1 shows the learning curves behind that row.

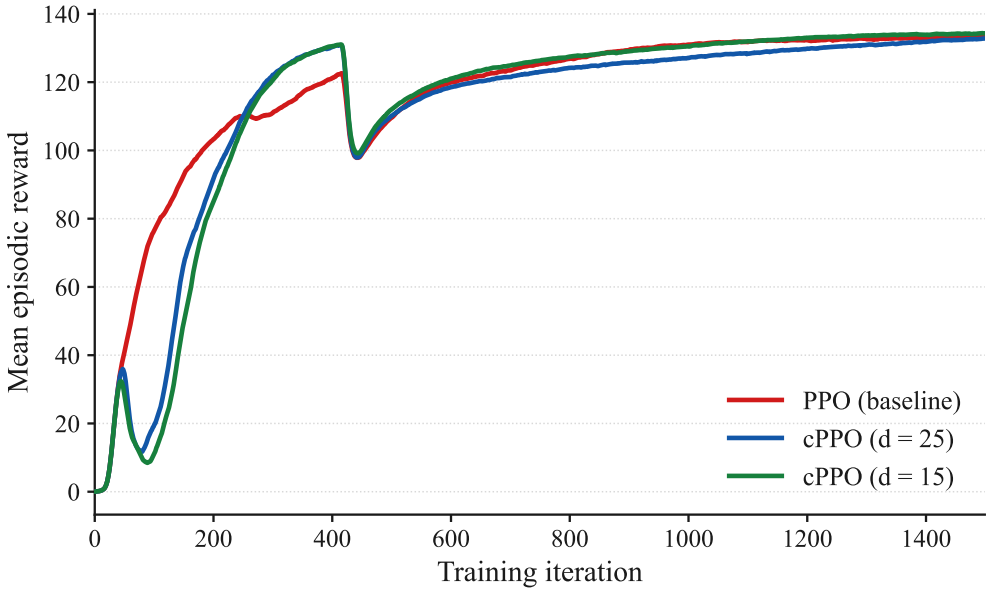

Figure 4.1. Mean episodic reward, averaged across the five seeds.

All three arms converge to the same level. The constrained arms learn more slowly for roughly the first two hundred iterations, which is the multiplier of Section 4.7 suppressing early exploration, and the gap closes well before training ends. Curves in this and every subsequent figure are exponentially smoothed for legibility; every value quoted in the text is computed from the unsmoothed logs.

The reward total is a sum of shaped terms, and the two that carry the task are plotted in Figure 4.2. Both converge to a common level on all three arms, so the equality of the reward row is not an artifact of one term compensating for another.

The soft-margin violation fractions in the lower rows are included for completeness and are deliberately not used as the safety headline. They count the fraction of control steps on which a continuous margin falls on the wrong side of a binary threshold, which exaggerates differences between policies whose margins differ only slightly, and they are measured on a still-exploring policy. Their dispersion illustrates the problem: the baseline's standard deviation of 0.338 is twice the constrained agent's mean. The rows are nevertheless informative in one respect that anticipates Section 4.6, in that the spread falls by roughly eightfold while the means separate by a factor of under three.

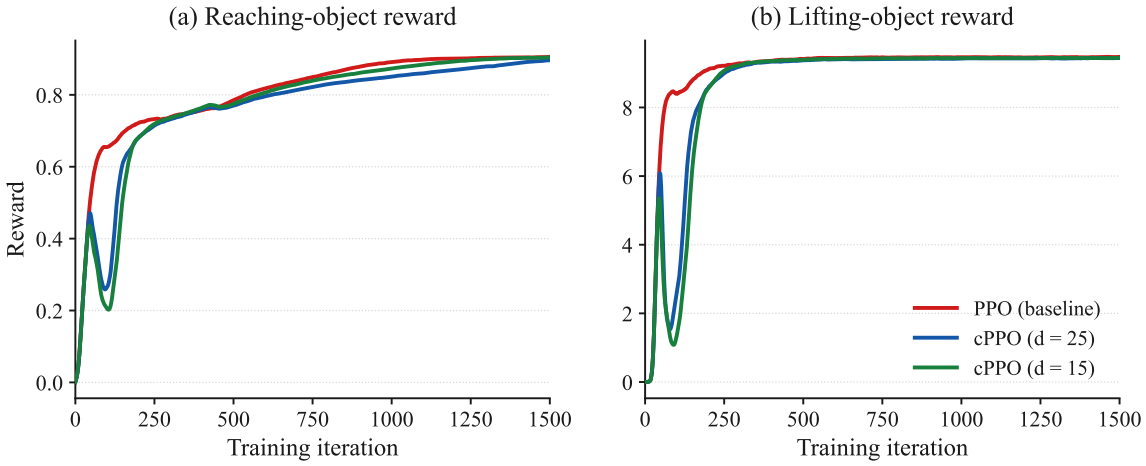

Figure 4.2. The two task-carrying reward terms, averaged across the five seeds. The arms are indistinguishable on lifting after roughly three hundred iterations, which is the training-time counterpart of the lift-success row of Table 4.3. Curves are exponentially smoothed for legibility.

Evaluation-time task performance is reported in Table 4.3. Following the audit's reporting rule, goal-reach is given as a distance distribution together with success at three thresholds, never as a single figure. The reason is structural rather than statistical. Under the weld abstraction described in Section 3.5, the cube's pose is the tool frame's pose once the weld latches, so "cube within tolerance of the goal" reduces to "tool frame within tolerance of the goal". A converged policy solves that on nearly every episode and a diverged one fails on nearly every episode, which drives any single-threshold success rate towards a ceiling and leaves it unable to rank two policies that have both converged. It was exactly this ceiling that produced the implausible 100 % against 0 % figures in the withdrawn version of this work.

Table 4.3. Evaluation-time task performance on the frozen deterministic policy. Each arm is 15,000 episodes (5 training seeds × 3 evaluation seeds × 1000). Bracketed figures are the standard deviation across the five training seeds.

Metric	PPO (baseline)	cPPO, $d = 25$	cPPO, $d = 15$
Lift success	99.91 % (0.09)	99.87 % (0.08)	99.90 % (0.08)
Goal-reach < 1 cm	93.03 % (7.52)	98.33 % (1.04)	**98.93 % (0.95)**
Goal-reach < 2 cm	98.78 %	99.58 %	**99.84 %**
Goal-reach < 5 cm	99.81 %	99.86 %	**99.90 %**
Goal distance, mean (m)	0.0062	0.0051	**0.0047**
Goal distance, median (m)	0.0050	0.0041	0.0041
Goal distance, p90 (m)	0.0087	0.0064	**0.0059**
Goal distance, max (m)	0.812	1.326	0.708

Every arm lifts the cube on essentially every episode, and the constrained arms are ahead of the baseline at every goal-reach threshold and across the whole distance distribution. The margins are small and should not be leaned on as evidence that the constraint improves the

task. The robust claim is the negative one, supported across seven independent measures: constraining the policy did not cost task performance, at either budget.

Figure 4.3 plots the success rows of that table twice. Panel (a) is the conventional view with the axis zoomed to the top of its range, since on a full scale from zero all twelve bars would be visually identical. Panel (b) plots the same numbers as failure rate on a logarithmic axis, where a difference between 99.91 and 93.03 % becomes a difference between 0.09 and 6.97 %, a factor of more than seventy. The second panel is the honest way to look at success rates this close to the ceiling, and it is the reason the goal-reach rows are reported at three thresholds rather than one.

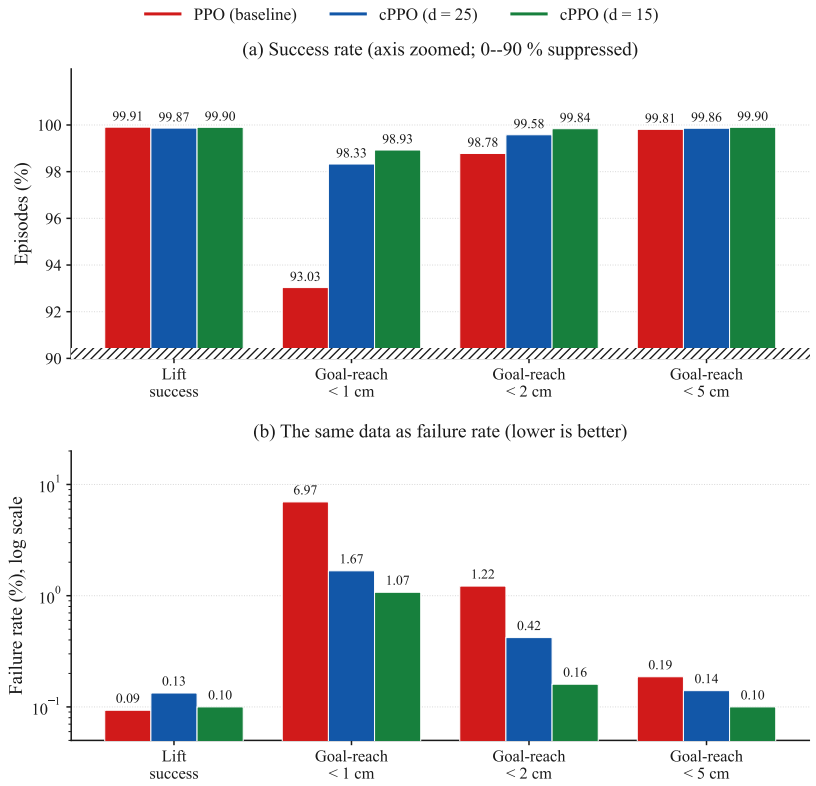

Figure 4.3. Evaluation-time task performance, 15,000 episodes per arm. Panel (a) shows success rates with the vertical axis zoomed to the 90 to 100 % band, the suppressed region marked by the hatched strip at the axis base. Panel (b) shows the complement, the failure rate, on a logarithmic axis, where differences invisible in panel (a) separate by more than a factor of seventy. Lower is better in panel (b).

One column of that table is more interesting than the margins, and it is the bracketed one. The seed-to-seed standard deviation of the sub-centimetre success rate falls from 7.52 percentage points on the baseline to 1.04 and 0.95 on the constrained arms, a sevenfold to eightfold tightening. The per-seed values behind the baseline figure run 80.00, 98.77, 94.83, 94.17 and 97.40 %, so one baseline seed in five lands roughly fifteen points below the others on task precision. Neither constrained arm has a seed below 96.97 %. The variance collapse that Section 4.6 reports for safety is therefore visible in task performance as well, which was not anticipated and is not something the constraint was designed to do.

Every arm produces a small number of catastrophic-miss episodes, with a maximum goal distance between 0.7 and 1.3 metres. These were not investigated and are recorded as an open item. They are noted here so that a reader encountering them in the per-episode data is not misled into treating them as an artifact of one arm.

4.5 Safety

Table 4.4 reports safety on the frozen policy, pooled over the 15,000 evaluation episodes per arm. Following the audit's reporting rule, the table leads with episodes that reached an actual kinematic singularity, meaning a manipulability measure [17] below 10^{-4}, which is a statement about the arm's genuine loss of a degree of freedom rather than about a calibrated soft margin.

Table 4.4. Safety on the frozen policy, pooled over 15,000 evaluation episodes per arm. Bracketed figures are the standard deviation across the five training seeds.

Metric	PPO (baseline)	cPPO, $d = 25$	cPPO, $d = 15$
True singularity crossings ($w < 10^{-4}$)	2.66 % (399) [3.79]	0.38 % (57) [0.36]	**0.07 % (10) [0.13]**
Mean episode-minimum w	0.0465	**0.0669**	0.0628
Worst single-episode w	0.000001	0.000001	0.000001
Joint limit touched at all	10.70 %	**0.00 %**	**0.00 %**
Collision touched at all	0.03 %	0.03 %	0.02 %
Episodic cost, mean	84.05	15.21	**11.31**
Episodic cost, p90	200.38	56.90	**43.77**
Episodic cost, max	343.01	**219.59**	228.51

The constraint reduces true singularity crossings by a factor of 7.0 at a budget of 25, from 399 episodes to 57 out of 15,000, and by a factor of 39.9 at a budget of 15, to 10 episodes. This is a stricter and more meaningful measurement than the soft-margin fraction of Table 4.2, and the effect survives the change of metric, which the withdrawn result did not. Figure 4.4 plots the four constraint outcomes of that table, each on its own scale, because they differ by three orders of magnitude and a shared axis would render three of the four flat.

The joint-limit row is the cleanest single result in the dataset. Joint-limit contact occurs in 10.70 % of baseline episodes and in none of either constrained arm's 15,000 episodes, on any seed. Three qualifications belong with it. First, this is an observed zero over a finite sample and not a proof of impossibility; the honest statement is that no joint-limit contact was observed, with the sample size given so that a reader can bound the claim themselves. Second, the result is notable precisely because the joint-limit term had been classified as inactive by construction until the recalibration of Section 3.9 reclassified it, so the budget is being spent across more than the single manipulability term that earlier write-ups of this project

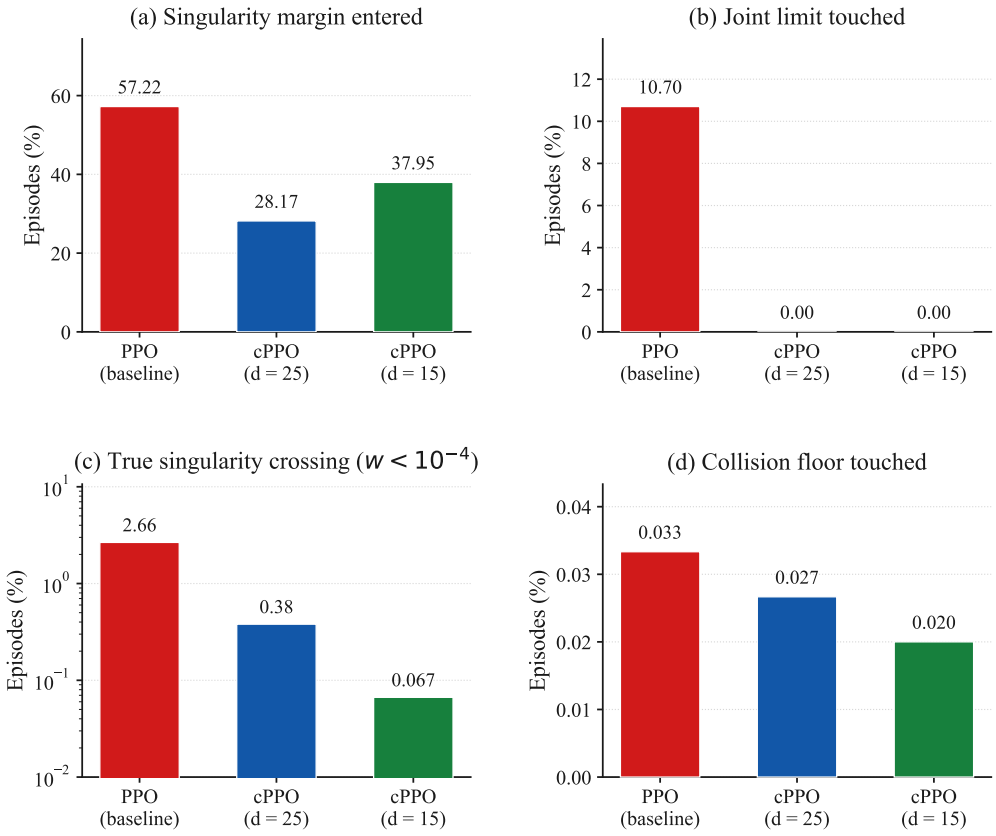

Figure 4.4. The four safety outcomes of Table 4.4, each panel independently scaled. Panel (c) uses a logarithmic axis. Note that (a) counts entry into the calibrated soft margin while (c) counts an actual loss of a degree of freedom, which is why the constrained arms improve far more on (c) than on (a): the constraint does not merely keep the arm out of the margin, it prevents the margin excursions from deepening into genuine singularities.

described. Third, the baseline's 10.70 % is itself a seed lottery rather than a stable property: the per-seed rates are 47.83, 0.00, 0.00, 0.00 and 5.67 %, so a single seed contributes almost all of it. That pattern is the subject of Section 4.6.

Figure 4.5 separates the training-time cost into its three terms and shows where the budget is actually spent. The joint-limit panel is the clearest: the baseline's contribution grows steadily from roughly iteration 700 onward while both constrained arms hold at zero for the whole run, which is the training-time counterpart of the evaluation row just discussed. The collision panel is flat at the vertical scale of a thousandth of a cost unit, confirming the term is inactive rather than merely small.

Collision was near zero for all three arms and remains so. It is reported for completeness and confirms that the collision floor is genuinely inactive at these settings.

One counter-result must be stated plainly rather than omitted because it is unflattering. The worst single-episode manipulability is identical for all three arms at 0.000001. Whatever the constraint does, it does not make the rare worst-case excursion shallower, and tightening the

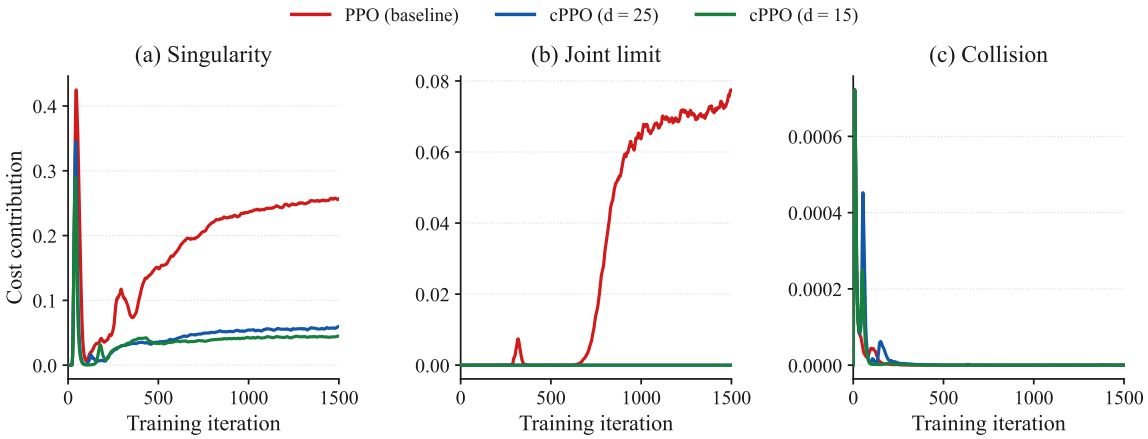

Figure 4.5. Episodic safety cost separated into its three terms, averaged across the five seeds. Note the differing vertical scales: the singularity term is an order of magnitude larger than the joint-limit term and two orders larger than collision. Curves are exponentially smoothed for legibility.

budget from 25 to 15 does not change that either. It reduces how often and how consistently the policy approaches a singularity, and the aggregate consequences of that are visible in the episodic-cost rows, where the worst episode costs 219.59 and 228.51 against the baseline's 343.01 and the ninetieth percentile falls from 200.38 to 56.90 and 43.77. The worst *episode* is less costly overall even though the worst *instant* is equally deep. A claim that constrained reinforcement learning bounds the depth of the worst singular excursion is not supported by this data and is not made. For a safety argument the distinction matters practically: on hardware, a policy that visits a near-singular configuration one seventh as often but just as deeply when it does still requires the same instantaneous protection, and buys its margin in expected exposure rather than in worst-case severity. This is the distinction Brunke *et al.* [14] draw between constraint satisfaction in expectation and a safety certificate, discussed in Section 2.6.

The mechanism behind that distinction is visible in Figure 4.6. The mean episode-minimum manipulability of the $d = 15$ arm settles roughly four times higher than the baseline's, so the typical episode operates further from the singular set. The worst instant, which no averaged curve can show, is unchanged at 0.000001 on all three arms.

Figure 4.7 closes the section with the quantity the budget is written against. The baseline's episodic cost climbs throughout training and settles near 82, more than three times its own budget, because nothing in its objective penalises it for doing so. Both constrained arms settle beneath their budgets and stay there.

4.6 The principal finding: collapse of seed-to-seed variance

The largest effect measured in this batch is not visible in any mean. It appears only when the five seeds are examined individually, which Table 4.5 does. That seed-to-seed spread is

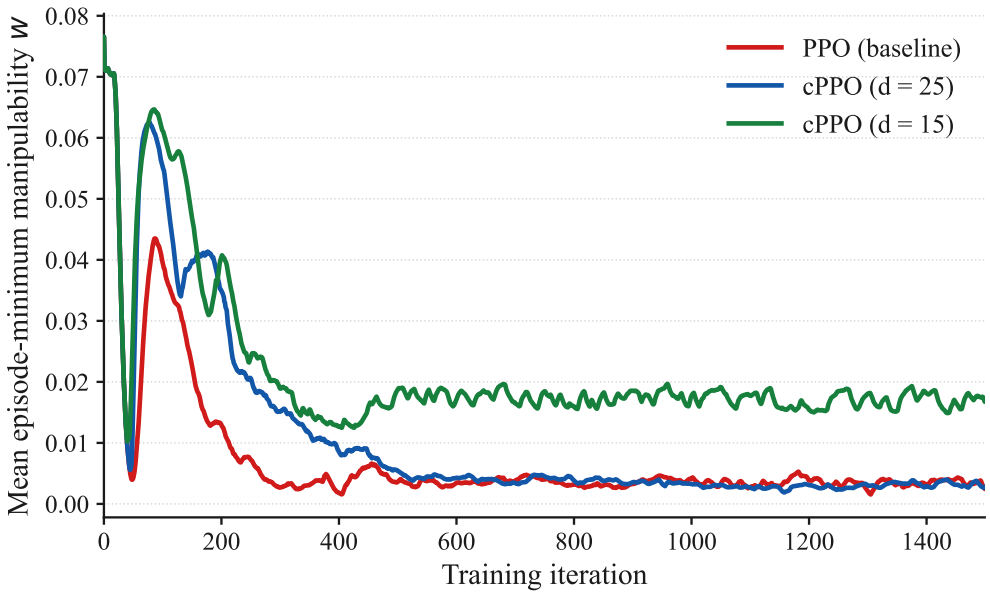

Figure 4.6. Mean episode-minimum manipulability w against training iteration, averaged across the five seeds. Higher is safer. The tighter budget holds the arm materially further from the singular set for the whole of the second half of training. Curves are exponentially smoothed for legibility.

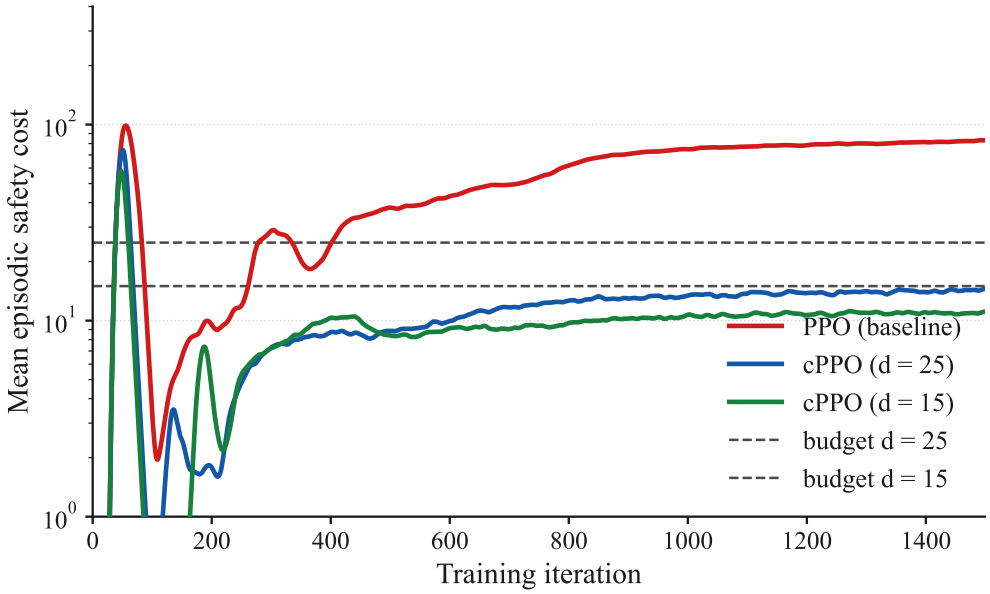

Figure 4.7. Mean episodic safety cost against training iteration, averaged across the five seeds, on a logarithmic axis. The dashed lines are the two budgets. The vertical axis is clipped below 1, which suppresses an early-training transient in the first two hundred iterations; the full range spans six decades and would compress the converged region into a sliver. Curves are exponentially smoothed for legibility.

itself a documented property of deep reinforcement learning rather than a peculiarity of this environment: Henderson *et al.* [6] show that random seed alone can produce statistically distinct outcome distributions for the same algorithm on the same task, and it is on that basis that this study reports dispersion across seeds rather than a single mean.

Table 4.5. Per-seed episodic cost, training tail mean and evaluation mean, with the peak Lagrange multiplier reached during training.

Quantity	s1	s3	s4	s52	s54	spread
Training, tail-mean episodic cost						
PPO (baseline)	102.10	162.30	30.04	106.93	7.95	20.4×
cPPO, $d = 25$	18.01	11.93	19.69	9.50	11.98	2.1×
cPPO, $d = 15$	14.08	11.61	3.83	10.65	14.53	3.8×
Evaluation, mean episodic cost over 3000 episodes						
PPO (baseline)	104.52	164.55	31.59	109.41	10.20	16.1×
cPPO, $d = 25$	20.20	12.04	21.39	8.91	13.50	2.4×
cPPO, $d = 15$	16.02	11.15	3.43	10.63	15.29	4.7×
Peak λ reached during training						
cPPO, $d = 25$	15.84	40.46	30.30	46.32	48.05	
cPPO, $d = 15$	17.62	35.47	40.83	27.13	38.56	

The same training data is plotted in Figure 4.8, where each seed contributes one vertical segment joining its three arm values. The figure is drawn this way, rather than as three sorted bands, because a sorted band would show the collapse in spread while hiding which direction each seed moved in. Both facts matter to the reading below.

Figure 4.9 shows the same collapse developing over training rather than at its endpoint, with every seed drawn separately. The baseline panel is the one to look at: five runs of the same algorithm in the same environment, differing only in the random seed, ending anywhere between 8 and 162. The constrained panels are not merely lower, they are narrow.

The baseline row is the behaviour of unconstrained policy optimisation with the cost merely observed and never acted upon, and it is extraordinarily inconsistent. Its natural episodic cost ranges from 7.95 on seed 54 to 162.30 on seed 3, a spread of more than twentyfold, for the identical algorithm in the identical environment differing only in the random seed. Seed 54 produces a policy that is accidentally safe. Seeds 3, 52 and 1 produce policies that are severely unsafe. Nothing in the training procedure distinguishes them and nothing in the training telemetry would warn an engineer which one they had obtained. The same lottery is visible in the evaluation column, and in the joint-limit rates of Section 4.5, where one seed touches a joint limit on 47.83 % of its episodes and three seeds never touch one at all.

Both constrained arms pull every seed into a narrow band beneath their budget irrespective

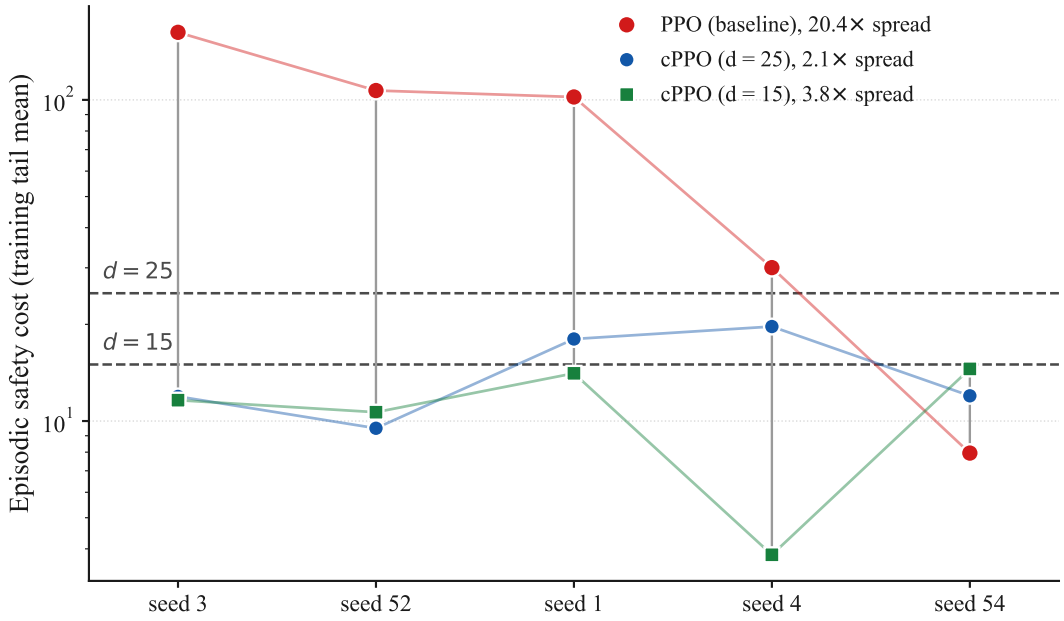

Figure 4.8. Per-seed episodic safety cost on a logarithmic scale. Each vertical segment joins the three arms for one seed, and the two horizontal lines are the budgets. Seeds are ordered by descending baseline cost rather than by seed number, so that the baseline's twentyfold sweep across the plot can be compared against the two constrained arms, which stay flat. The spread beside each legend entry is the ratio of that arm's highest seed to its lowest.

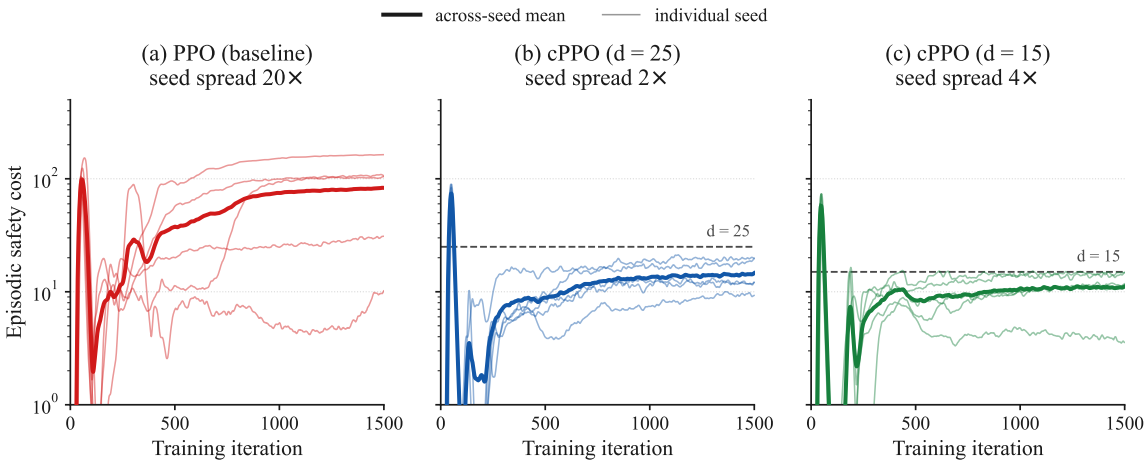

Figure 4.9. Training-time episodic cost for every seed, one panel per arm, on a shared logarithmic scale. Thin lines are individual seeds and the thick line is their mean. The spread quoted above each panel is the ratio of the highest to the lowest seed at the end of training. Curves are exponentially smoothed for legibility.

of where its unconstrained counterpart sat. Seed 3, whose baseline counterpart is the worst in the batch at 162.30, ends at 11.93 and 11.61.

This should not be read as a uniform improvement, and the per-seed data makes the qualification unavoidable, though it is a narrower qualification than the previous version of this chapter reported. The band is entered from both directions, but on one seed of five rather than on most of them. On seed 54, whose unconstrained policy was already comfortably under budget at 7.95, both constrained arms finish *higher*, at 11.98 and 14.53 in training and 13.50 and 15.29 in evaluation. On the other four seeds the cost falls sharply. What the constraint supplies is therefore best described as a ceiling rather than a reduction applied to every run: it prevents the disastrous outcomes without guaranteeing that a fortunate run stays as fortunate as it would have been. Since which of the two a given training run will produce is not knowable in advance, which is precisely the lottery described above, trading an unpredictable draw between 7.95 and 162.30 for a reliable band around 12 is the correct trade for a system intended to run on hardware. It remains a trade, and it is presented as one.

The effect is confirmed independently on held-out evaluation episodes rather than on training rollouts. Mean episodic cost falls from 84.05 to 15.21 and 11.31, reductions of 81.9 % and 86.5 %. The more important quantity is the standard deviation across seeds, which falls from 62.75 to 5.38 and 5.01, a tightening of 11.7 and 12.5 times, and a larger proportional change than the change in the mean. The baseline's standard deviation of 62.75 is itself smaller than its own mean of 84.05 only narrowly, which is the formal signature of the lottery described above.

The engineering reading is that unconstrained optimisation on this task does not have a safety level. It has a distribution of safety levels, and which one is drawn is close to arbitrary. The constrained agent's contribution in this batch is to make the outcome predictable, at no measurable task cost. For a manipulator intended to run on real hardware this is arguably the more useful property of the two: a mean improvement tells an integrator what to expect on average across training runs they will never perform, whereas a variance collapse tells them what to expect from the single policy they actually trained. A safety argument that depends on having drawn a fortunate seed is not a safety argument.

4.7 The multiplier, measured rather than inferred

The previous version of this chapter recorded only the value of λ at the final iteration and inferred the rest of its behaviour. The full per-iteration trajectories have since been extracted from the training logs, and Figure 4.10 plots them. The inference is confirmed and can now be stated as a measurement.

The pattern is the same on every seed and at both budgets. The multiplier sits at zero for roughly the first forty iterations while the policy is still learning to reach at all and its episodic

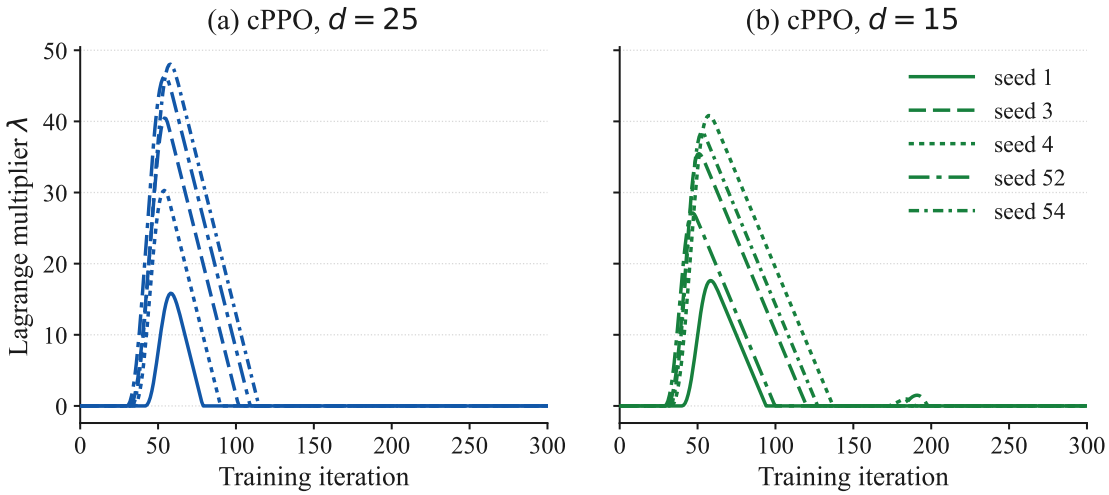

Figure 4.10. Lagrange multiplier against training iteration, all five seeds, both constrained arms. The multiplier is zero for the remaining 1200 iterations in every run.

cost is low for want of competence. It then rises steeply as the policy becomes capable enough to exceed the budget, reaching a single sharp peak between iterations 47 and 58, and decays back to zero once the cost has been driven under the budget. Peak values run from 15.84 to 48.05 at $d = 25$ and from 17.62 to 40.83 at $d = 15$, and λ is above 0.01 for between 36 and 120 iterations of the 1500. Nine of the ten runs end with the multiplier at exactly zero; the tenth, seed 1 at $d = 15$, ends at 0.053.

Two things follow. The first is that a final λ of zero says nothing about whether the constraint was active, which is why the previous version's table of final values was close to uninformative. On every seed the constraint engaged hard and then relaxed, and reading the final value alone would have suggested that it never engaged at all. The second is that the shape is the one the optimiser's documented dynamics predict rather than an interpretation invented to fit these numbers. Plain dual ascent on the multiplier is integral control on the constraint, and an integral controller with no proportional or derivative action overshoots and then unwinds, which is precisely the behaviour Stooke *et al.* [13] analyse and the reason they propose adding proportional and derivative terms. Section 2.5 sets this out, and Chapter 6 identifies the PID form as the natural extension of this work. The single clean overshoot visible in Figure 4.10, rather than the sustained oscillation of their Figure 1, reflects that the cost here is driven under budget and stays there, so the controller has nothing further to correct.

4.8 The effect of tightening the budget

The two constrained arms differ in one number, so the comparison between them isolates the effect of how hard the constraint is applied. Section 2.11 identifies this as a gap in the published work, where the budget of 25 is inherited and never varied.

Tightening from 25 to 15 improves nearly every safety measure and costs nothing measurable. Singularity crossings fall further, from 57 episodes to 10, so the tighter budget removes another 82 % of what the looser one left. Mean episodic cost falls from 15.21 to 11.31 and the ninetieth percentile from 56.90 to 43.77. Joint-limit contact was already at zero and stays there. On the task side the tighter arm is marginally the best of the three at every goal-reach threshold and carries the highest training reward at 134.09, so no trade-off against task performance is visible at this budget.

Three qualifications keep that from being read as a general rule. The tighter budget does not improve the soft-margin singularity fraction during training, which rises from 0.169 to 0.233, and the two measures disagree because one is taken on an exploring policy against a calibrated margin and the other on a frozen policy against a true rank deficiency. It does not improve the worst single-instant manipulability, which is unchanged. And the seed-to-seed spread of episodic cost is slightly wider at $d = 15$ than at $d = 25$ in relative terms, at 4.7 against 2.4 times, because the tighter budget drives one seed down to 3.43 while another sits at 15.29. The defensible statement is that within the range tested, tightening the budget continued to help and did not begin to cost task performance. Whether that continues below 15 is untested, and Section 4.9 records it as such.

4.9 Limitations

Four limitations bound what this chapter establishes, and they are stated here in full rather than distributed as caveats.

The first is the seed count. Five seeds is enough to demonstrate that the outcome of unconstrained training is a lottery and that the constraint closes the spread, and it is consistent with the practice Henderson *et al.* [6] describe as a minimum. It is not enough to put a confidence interval on the size of the effect. Every dispersion figure in this chapter should be read as an estimate from five draws.

The second is that no off-policy arm was trained. The `sac` arm registered in the original plan is absent, so the algorithm-family question examined in comparable work on manipulation tasks [16] remains open in this setting. What this batch does answer, and what the previous version of this chapter listed as its principal missing piece, is the effect of an actively binding budget: at $d = 15$, which is below the baseline's natural operating cost on every seed, the constraint binds throughout and Section 4.8 reports the result.

The third is a pair of data-hygiene faults found during analysis, filtered around, and not repaired at source. Three superseded pre-audit constrained runs from the gradient-clip-bug era still occupy the same experiment directory as the current runs under identical labels; the results reported here exclude them by checkpoint-path date, and the evaluation script's checkpoint selection was verified to resolve to the newer run, but both protections rest on

file metadata rather than on the stale runs having been removed. Separately, the append-only evaluation results file was found to carry stale rows from a superseded evaluation sweep, which were filtered by checkpoint path before analysis. Neither fault affected the numbers reported here, since both were caught and excluded, but both are standing risks to any future automated aggregation. They are documented rather than quietly fixed because the discipline of recording near-misses is part of this project's method, and because the entire correction that produced this chapter began as exactly such a near-miss that was not caught in time.

The fourth is the counter-note of Section 4.5, restated because it is the sharpest limit on the safety claim itself. The worst single-episode manipulability is identical across all three arms. The constraint demonstrably reduces the frequency and the seed-to-seed consistency of near-singular operation, and it reduces the cost of the worst episode, but it does not reduce the depth of the worst instant, and tightening the budget does not either. Any hardware safety case built on this result must rest on expected exposure rather than on worst-case severity, and must retain whatever instantaneous protection it would otherwise have required.

4.10 Summary

Four claims are established by this batch.

First, the control arm reproduces the unconstrained baseline exactly, to identical stored weights, on all five seeds, and again independently across all 15,000 evaluation episodes. That confirms the gradient-clipping correction and licenses the constrained-versus-baseline comparison to be read directly, because the artifact term of Equation (4.1) is exactly zero rather than merely small.

Second, the constraint costs no measurable task performance at either budget. Reward differs by less than the seed-to-seed standard deviation and the tighter arm ends marginally above the baseline; goal-reach is equal or better at every threshold and across the whole distance distribution.

Third, on safety, the constraint reduces true singularity crossings by a factor of 7.0 at a budget of 25 and 39.9 at a budget of 15, and eliminates all observed joint-limit contact at both. Its largest measured effect, however, is on variance rather than on the mean. It collapses a twentyfold seed-to-seed spread in natural episodic cost to roughly two, and a standard deviation of 62.75 across seeds to about 5, converting an outcome that was close to a lottery into a predictable one. The same collapse appears unexpectedly in task precision, where the spread of sub-centimetre success falls from 7.52 points to about 1.

Fourth, the multiplier trajectories show a single sharp engagement peaking near iteration 50 and decaying to zero, on every seed and at both budgets, which is the behaviour plain dual ascent is known to produce and confirms by measurement what the previous version of this

chapter could only infer.

The comparison this project registered in advance as its central claim, an actively binding budget against the control arm, is answered here by the $d = 15$ arm, and tightening the budget was found to help further at no task cost within the range tested. What is not answered is the off-policy comparison, and what remains open is whether the benefit continues below a budget of 15. Alongside those findings sits a methodological one: a previously reported effect of this kind in this project was an implementation artifact, the artifact has been removed, its removal has been verified as strongly as such a thing can be verified, and the corrected comparison still shows a substantial and differently shaped safety benefit. Reporting that sequence honestly is a more defensible contribution than the original headline it replaces [7].

CHAPTER 5

RELATION WITH A REAL-WORLD PROBLEM

The manipulator studied in this thesis is not a laboratory abstraction. The UR5e is a commercially deployed collaborative arm, and machines in its class already do pick-and-place, machine tending, small-parts assembly, and camera-inspection work on production floors, as Chapter 1 describes. Safety on this class of arm is not a concern raised for the purposes of this thesis; it is an active requirement wherever the arm shares a workspace with a person rather than working behind a fence. The specific hazard addressed here, a manipulator configuration drifting toward a kinematic singularity or a joint limit during a reach-and-grasp motion, is exactly the kind of behaviour that is invisible to a person watching the arm move and only becomes apparent as a loss of control authority or an unexpected fault.

The engineering contribution of this thesis is not a new algorithm. Both PPO and PPO-Lagrangian are established methods, reviewed and specified in Chapters 2 and 3. What this thesis contributes is a way of stating the safety requirement that an integrator, not only a researcher, can work with directly. A reward-shaped safety term is a weight chosen by trial and error and folded into a single scalar that also carries the task reward, so nobody outside the training run can read off from it what the arm was actually asked to respect. A constraint is a number, the episodic cost budget d. It can be set from a measurement of the hardware and checked afterward against what the deployed policy actually did. Chapter 4 shows what that check looks like: at $d = 25$ the trained policy touched a joint limit in 0.00 % of 15,000 evaluation episodes, against 10.70 % for the unconstrained baseline, and reduced true singularity crossings by a factor of 7.0; tightening the same budget to $d = 15$ reduced singularity crossings by a factor of 39.9 and left joint-limit contact at zero. None of these are properties a reward weight can be audited against after the fact, because a reward weight was never a statement of a limit to begin with.

The result most relevant to an integrator, though, is not the headline safety numbers but the one in Section 4.6. Training the unconstrained baseline five times, changing nothing but the random seed, produced episodic safety costs ranging from 7.95 on one seed to 162.30 on another, a spread of more than twentyfold, with a matching lottery in joint-limit contact: one seed touched a joint limit on 47.83 % of its episodes, three never touched one at all. An integrator does not train five policies and average across them. They train one, and that one is whichever draw the random seed produced. The constrained agent pulled every seed into a band roughly two to five times as wide rather than twenty, at no measured cost to task performance. A stated budget therefore buys something beyond a lower safety cost on

average: the ability to say in advance what the shipped policy is likely to do, rather than finding out only after it has already been trained and deployed.

None of this should be read as a claim about a deployed system. Nothing in this thesis ran on the physical UR5e; the sim-to-real step is future work, discussed in Chapter 6. This chapter argues what the method makes possible, not what has been demonstrated on hardware, and that distinction has to hold for the strongest safety claim here as much as any other. Section 4.5 reports a counter-result alongside the reductions above: the worst single-episode manipulability measured on any arm, constrained or not, is identical. The constraint reduces how often the policy nears a singularity and how costly the typical episode is, but it does not make the rare worst-case excursion any shallower, at either budget. A hardware safety case built on this method would have to rest on expected exposure, the frequency and typical severity of near-singular operation, rather than on a claim that the worst case has been bounded, and it would still need whatever instantaneous protection a deployed system requires [14].

That predictability carries a socio-economic argument beyond the engineering one. A collaborative arm is attractive to a smaller manufacturer because it can run without a fenced cell or a dedicated safety engineer, which is also why UR-class arms are common outside large firms with in-house robotics expertise. A safety requirement expressed as an auditable number fits that market: a specification an integrator can check against a policy's evaluation record is one a smaller operation can still verify, rather than one that requires reverse-engineering what a reward function was actually trained to trade off. Reducing how often an arm approaches a joint limit or a near-singular configuration also reduces a source of unplanned downtime and mechanical wear. None of this rests on the specific numbers measured here; it follows from stating the requirement as a constraint at all.

Mapped against the United Nations Sustainable Development Goals, this work sits most defensibly under two. SDG 9, Industry, Innovation and Infrastructure, is the direct fit: this is a method for making automation on already-deployed industrial hardware more auditable and more predictable, which is the kind of upgrade Target 9.4 asks industry to make. SDG 8, Decent Work and Economic Growth, applies through Target 8.8, which calls for safe working environments for all workers: the entire premise of a collaborative arm is that it shares space with a person, and a policy whose safety behaviour is stated as a checkable budget rather than hidden inside a reward weight is a step toward being able to certify that shared space as safe. Other goals in this family, such as SDG 12 on responsible production, would need a claim about resource use or waste that this thesis does not make, and are left out rather than listed for completeness.

CHAPTER 6

CONCLUSIONS AND FUTURE WORKS

6.1 Conclusion

This thesis set out to determine whether stating a manipulability safety requirement as an explicit constraint, rather than folding it into the reward, changes the safety behaviour of a learned grasping policy on a UR5e manipulator, and whether it changes how predictable that behaviour is across independent training runs (Section 1.3). Both questions are answered. Constraining the policy reduces unsafe behaviour, and its larger and less anticipated effect is on how consistently that behaviour is reproduced from one training run to the next (Section 4.10). The six specific objectives set out at the start of this thesis were each met in turn.

The first objective was to implement an unconstrained PPO baseline and a constrained PPO-Lagrangian agent for a UR5e pick-and-lift grasping task in Isaac Lab, trained under a shared floor on manipulability. Both were built and trained as specified, on the frozen weld environment described in Chapter 3, at the calibrated thresholds recorded in Section 4.1.

The second objective was to isolate the effect of the constraint itself from the effect of the extra cost critic a Lagrangian method carries, by training a control arm whose multiplier is pinned at zero throughout. That isolation succeeded more completely than the design anticipated: Section 4.2 found the control arm's stored network weights identical to the unconstrained baseline's, on every seed, so the artifact term in the decomposition of Equation (4.1) is zero by direct verification rather than by assumption, and every safety difference reported in this thesis can be attributed to the constraint alone.

The third objective was to establish whether the constrained agent's behaviour depends on how tight its budget is, by training it at two episodic cost budgets, 25 and 15, rather than one. Section 4.8 answers it: tightening the budget continued to help within the range tested, reducing true singularity crossings further and leaving joint-limit contact at zero, at no measurable cost to task performance, though the worst single-instant manipulability was unmoved by the tighter budget as much as by the looser one.

The fourth objective was to train every arm on five independent seeds and report outcomes as a distribution rather than a single mean. That is the reporting convention followed throughout Chapter 4, and it is what exposed the finding the general objective was actually asking about: the unconstrained baseline's safety outcome is not one number but a distribution spanning

a factor of twenty, and no telemetry available during training would have told an engineer which end of that distribution a given run had landed on.

The fifth objective was to evaluate every trained policy on frozen, held-out episodes, so the reported figures describe the policy that would actually be deployed rather than one still exploring. Section 4.1 sets out that protocol, forty-five thousand evaluation episodes across three held-out seeds, and it exists because an earlier version of this project's evaluation had read safety statistics from a still-learning policy and reported a difference that later proved to be measuring an implementation artifact rather than the constraint (Section 4.2). The corrected protocol is what makes every other objective's finding trustworthy.

The sixth and final objective was to determine whether the constraint changes the average safety outcome of a training run, its spread, or both. It changes both, but not by equal amounts. On the mean, the constrained agent reduces true singularity crossings by a factor of 7.0 at a budget of 25 and 39.9 at a budget of 15, and eliminates all observed joint-limit contact at both, against 10.70 % of baseline episodes touching one. On the spread, which Section 4.6 identifies as the larger effect, a twentyfold seed-to-seed range in episodic safety cost collapses to roughly two-to-fivefold, and the same collapse turns up unprompted in task precision, where the seed-to-seed standard deviation of sub-centimetre success falls from about 7.5 percentage points to about one. Neither effect cost the constrained agent anything on the task itself: reward and goal-reach are equal to or better than the baseline's at both budgets.

Read together, the six objectives support the general objective's answer directly. A stated safety budget does change what a learned grasping policy does, and its main effect is not a modest average improvement but the near-elimination of a lottery in which an unconstrained training run might, by the luck of its random seed alone, produce a policy that touches a joint limit on nearly half its episodes. Whether that policy is the one shipped to hardware is not something an unconstrained training procedure lets an engineer know in advance. A constrained one does.

6.2 Future Works

Five extensions follow from what this thesis chose not to attempt or could not finish, each positioned here rather than apologised for.

The most direct is closing the visual loop. This thesis trained on privileged object pose, the cube's true position handed to the policy directly, because isolating the safety comparison from a perception problem was the point of Layer 1 (Section 1.4). The registered programme's second layer would replace that privileged pose with an eye-in-hand RGB-D camera and object detector, and use the learned policy to tune the image Jacobian relating pixel motion to joint motion rather than fixing it in advance, extending the fixed, hand-derived

visual servoing loop of the predecessor project in this laboratory [4] into a learned, adaptive one. Whether a constraint framed on manipulability continues to hold once the arm is also reacting to a noisy, partially occluded camera feed is not something this thesis's simulation-only, full-state-observation result can answer, and is the natural next question.

The second is sim-to-real transfer to the physical UR5e, the programme's third layer. The trained policy already exports to both JIT and ONNX from the same checkpoint format used throughout this thesis, so the software path to a ROS 2 Humble deployment exists. What does not yet exist is a resolution to the largest known transfer gap: the simulation grasps with a lumped-mass abstraction of a Robotiq 2F-85, welding the cube to the tool frame at the moment of contact rather than modelling the grip physically, while the laboratory's real arm carries a ROBOTIS RH-P12-RN, a different gripper with different kinematics. Closing that gap is gripper-specific engineering, not a parameter to randomise away, and it is Layer 3's opening task rather than an afterthought to it.

The third is the comparison this thesis registered but did not complete. The plan called for an off-policy arm, soft actor-critic, alongside the on-policy PPO and PPO-Lagrangian agents actually trained; it was cut on schedule grounds, not because a reason emerged to expect it would behave differently. Continuous-control grasping has been demonstrated with both algorithm families [16], so nothing here argues against completing the comparison, only that time ran out before it could be run under the same fairness protocol as the rest of this batch.

The fourth is a specific and already-evidenced improvement to the constrained optimiser itself. Section 4.7 measured the Lagrange multiplier overshooting on every seed at both budgets before settling, a single sharp rise peaking between iterations 47 and 58 followed by a decay to zero. That is the textbook signature plain dual ascent is known to produce, integral control with nothing to damp it, and it is exactly the problem Stooke *et al.* [13] built PID-Lagrangian to correct, by adding proportional and derivative terms to the multiplier update. Because this thesis measured the overshoot rather than merely suspecting it, PID-Lagrangian is not a speculative extension here. It is a direct response to a documented result in Section 4.7, and the natural successor to the plain dual-ascent update used throughout this work.

The fifth is scale in the one dimension this thesis deliberately kept small: the number of seeds. Five seeds were enough to show that unconstrained training on this task produces a wide and unpredictable range of safety outcomes and that the constraint narrows it (Section 4.6), because the effect is large enough to appear clearly even at that sample size. Five seeds are not enough to attach a confidence interval to how large the narrowing is, a limitation Section 4.9 states directly. A larger seed count, beyond what this thesis's schedule allowed, would turn a demonstrated effect into a quantified one.

BIBLIOGRAPHY

[1] Weidong Xia, Yuqian Lu, Wenjun Xu, and Xun Xu. Deep reinforcement learning based proactive dynamic obstacle avoidance for safe human-robot collaboration. *Manufacturing Letters*, 41:1246–1256, 2024.

[2] Universal Robots A/S. UR5e technical specifications. Product datasheet, August 2023.

[3] Henrique C. Ferreira and Ramiro S. Barbosa. Deep reinforcement learning for adaptive robotic grasping and post-grasp manipulation in simulated dynamic environments. *Future Internet*, 17(10):437, 2025.

[4] Md Masrul Khan. *Enhancing Manipulator Control Through Image-Based Visual Servoing Techniques Using ROS2*. BSc thesis, Dept. of Mechatronics Engineering, Khulna University of Engineering & Technology, Khulna, Bangladesh, December 2025.

[5] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proc. 34th Int. Conf. Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 22–31. PMLR, 2017.

[6] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1):3207–3214, 2018.

[7] Fawad Khan, Wei Feng, Tianlun Huang, Zhiyong Wang, Xiao Liu, Shahid Asad Ali, and Weijun Wang. Reinforcement learning for precision grasping and safety-critical coordination in a robotic arm. *Intelligent Service Robotics*, 19(16), 2026.

[8] Alex Ray, Joshua Achiam, and Dario Amodei. Benchmarking safe exploration in deep reinforcement learning. Technical report, OpenAI, 2019.

[9] Javier García and Fernando Fernández. A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research*, 16(42):1437–1480, 2015.

[10] Shangding Gu, Long Yang, Yali Du, Guang Chen, Florian Walter, Jun Wang, and Alois Knoll. A review of safe reinforcement learning: Methods, theories, and applications. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12):11216–11235, 2024.

[11] Eitan Altman. *Constrained Markov Decision Processes*. Chapman and Hall/CRC, Boca Raton, FL, USA, 1999.

[12] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[13] Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. In *Proc. 37th Int. Conf. Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 9133–9143. PMLR, 2020.

[14] Lukas Brunke, Melissa Greeff, Adam W. Hall, Zhaocong Yuan, Siqi Zhou, Jacopo Panerati, and Angela P. Schoellig. Safe learning in robotics: From learning-based control to safe reinforcement learning. *Annual Review of Control, Robotics, and Autonomous Systems*, 5:411–444, 2022.

[15] Íñigo Elguea-Aguinaco, Antonio Serrano-Muñoz, Dimitrios Chrysostomou, Ibai Inziarte-Hidalgo, Simon Bøgh, and Nestor Arana-Arexolaleiba. A review on reinforcement learning for contact-rich robotic manipulation tasks. *Robotics and Computer-Integrated Manufacturing*, 81:102517, 2023.

[16] Asad Ali Shahid, Dario Piga, Francesco Braghin, and Loris Roveda. Continuous control actions learning and adaptation for robotic manipulation through reinforcement learning. *Autonomous Robots*, 46:483–498, 2022.

[17] Tsuneo Yoshikawa. Manipulability of robotic mechanisms. *The International Journal of Robotics Research*, 4(2):3–9, 1985.

[18] Yue Shen, Qingxuan Jia, Zeyuan Huang, Ruiquan Wang, Junting Fei, and Gang Chen. Reinforcement learning-based reactive obstacle avoidance method for redundant manipulators. *Entropy*, 24(2):279, 2022.

[19] Jiaming Ji, Borong Zhang, Jiayi Zhou, Xuehai Pan, Weidong Huang, Ruiyang Sun, Yiran Geng, Yifan Zhong, Juntao Dai, and Yaodong Yang. Safety gymnasium: A unified safe reinforcement learning benchmark. In *Advances in Neural Information Processing Systems 36 (NeurIPS), Datasets and Benchmarks Track*, 2023.

[20] Mayank Mittal, Calvin Yu, Qinxi Yu, Jingzhou Liu, Nikita Rudin, David Hoeller, Jia Lin Yuan, Ritvik Singh, Yunrong Guo, Hammad Mazhar, Ajay Mandlekar, Buck Babich, Gavriel State, Marco Hutter, and Animesh Garg. Orbit: A unified simulation framework for interactive robot learning environments. *IEEE Robotics and Automation Letters*, 8(6):3740–3747, 2023.

[21] Viktor Makoviychuk, Lukasz Wawrzyniak, Yunrong Guo, Michelle Lu, Kier Storey, Miles Macklin, David Hoeller, Nikita Rudin, Arthur Allshire, Ankur Handa, and

Gavriel State. Isaac gym: High performance GPU-based physics simulation for robot learning. *arXiv preprint arXiv:2108.10470*, 2021.

[22] Joshua Achiam. Spinning up in deep reinforcement learning. OpenAI, 2018. Accessed 3 August 2026.

[23] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proc. 5th Conf. Robot Learning (CoRL)*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100. PMLR, 2022.